

In24-S2-CS1050

4-bit Nano Processor Implementation

Lab Report



Group: Group 33

Team Members

UMAYANGA P.H.A. (240673E)

UTHTHARASENA U.G.N.S. (240675L)

VITHANAGE K.O. (240688E)

THISARUKA K.K.M. (240668T)

Department of Computer Science and Engineering

University of Moratuwa

Table of Contents

	Page No
1. Introduction.....	05
2. Objective.....	07
3. Assembly Program & Machine Code.....	08
4. Nano Processor Design Schematic.....	09
5. Synthesis Report Summary.....	10
6. Slow Clock	11
6.1. VHDL Code.....	
6.2. Testbench.....	
6.3. Timing Diagram.....	
7. 4bit Adder	14
7.1. VHDL Code.....	
7.2. Testbench.....	
7.3. Timing Diagram.....	
8. Arithmetic Unit	17
8.1. VHDL Code.....	
8.2. Testbench.....	
8.3. Timing Diagram.....	
9. 4bit Register	21
9.1. VHDL Code.....	
9.2. Testbench.....	
9.3. Timing Diagram.....	
10. Register Bank	25
10.1. VHDL Code.....	
10.2. Testbench.....	
10.3. Timing Diagram.....	
11. 2-to-1 Multiplexer	32
11.1. VHDL Code.....	
11.2. Testbench.....	
11.3. Timing Diagram.....	
12. 8-to-1 Multiplexer	35

12.1.	VHDL Code.....	
12.2.	Testbench.....	
12.3.	Timing Diagram.....	
13.	3-to-8 Decoder	38
13.1.	VHDL Code.....	
13.2.	Testbench.....	
13.3.	Timing Diagram.....	
14.	Program ROM	41
14.1.	VHDL Code.....	
14.2.	Testbench.....	
14.3.	Timing Diagram.....	
15.	Program Counter	44
15.1.	VHDL Code.....	
15.2.	Testbench.....	
15.3.	Timing Diagram.....	
16.	Program Incrementer	47
16.1.	VHDL Code.....	
16.2.	Testbench.....	
16.3.	Timing Diagram.....	
17.	Load Selector	49
17.1.	VHDL Code.....	
17.2.	Testbench.....	
17.3.	Timing Diagram.....	
18.	Instruction Decoder	52
18.1.	VHDL Code.....	
18.2.	Testbench.....	
18.3.	Timing Diagram.....	
19.	7 Segment Display	56
19.1.	VHDL Code.....	
19.2.	Testbench.....	
19.3.	Timing Diagram.....	
20.	Processor	59
20.1.	VHDL Code.....	
20.2.	Testbench.....	

20.3.	Timing Diagram.....	
21.	NanoProcessor.....	68
21.1.	VHDL Code.....	
21.2.	Testbench.....	
21.3.	Timing Diagram.....	
22.	Resource Optimization Strategies.....	72
23.	Additional Features	74
24.	Team Members — Individual Contributions.....	75
25.	Conclusion.....	76

1. Introduction

This laboratory project focuses on the design and implementation of a simple 4-bit nanoprocessor using VHDL and fundamental digital logic design principles. The nanoprocessor is designed to execute a limited instruction set consisting of MOVI, ADD, NEG, and JZR instructions, which enable basic arithmetic operations, data movement, and conditional branching. The main objective of this lab is to gain a practical understanding of how a processor functions internally by designing and integrating several important hardware components, such as a 4-bit add/subtract unit, multiplexers, program counter, register bank, instruction decoder, and program ROM. Through this project, students are able to study the interaction between the datapath and control unit while learning how instructions are fetched, decoded, and executed within a processor architecture.

In addition to processor design concepts, this lab also strengthens knowledge of combinational and sequential logic circuits, buses, memory structures, and hardware description using VHDL. Each component is individually designed, simulated, and tested before being integrated into the complete nanoprocessor system. The processor is then programmed using machine code stored in ROM to execute a simple assembly program. The final design is verified through simulation and implemented on the BASYS 3 FPGA development board, where outputs such as register values and status flags are displayed using LEDs and seven-segment displays.

Furthermore, this project emphasises the importance of teamwork and collaborative engineering practices. Since the lab is completed as a group project, team members are required to divide responsibilities, communicate effectively, and integrate independently developed modules into a fully functioning system. Overall, the lab provides valuable hands-on experience in computer organization, digital system design, FPGA implementation, and hardware-level problem solving, helping students understand the fundamental principles behind modern processors and embedded systems.

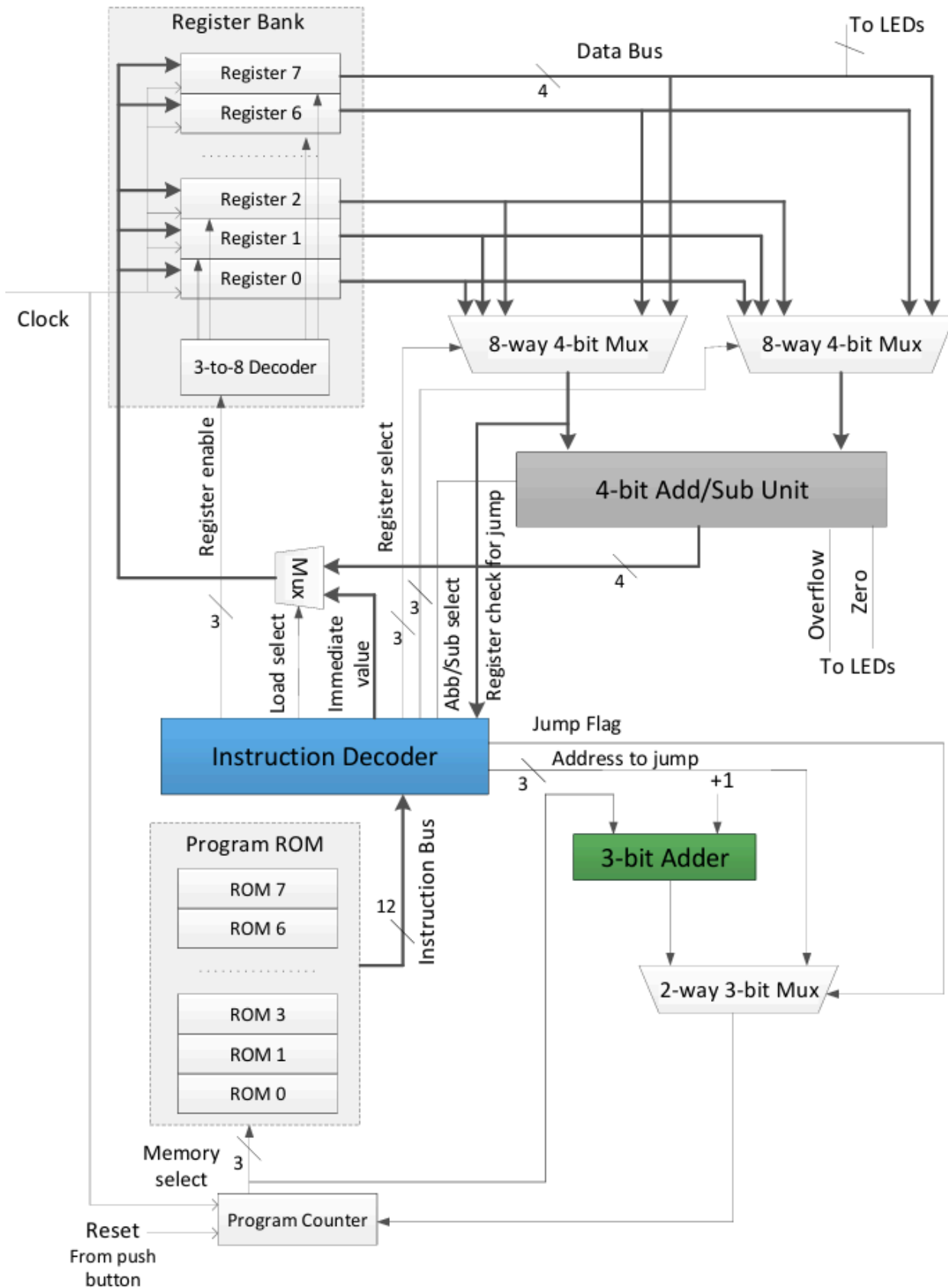


Figure 1 – High-level diagram of the nanoprocessor.

2. Objective

The task is to design and implement a functional 4-bit Nano Processor that executes a hardcoded sequence of 12-bit machine instructions. The processor must include: a 4-bit Add/Sub unit (signed arithmetic), an 8×4 Register Bank with enable signals, a 3-bit Program Counter with synchronous reset, k-way multiplexers for data routing, a Program ROM acting as an instruction LUT, and an Instruction Decoder that converts 12-bit opcodes into control signals.

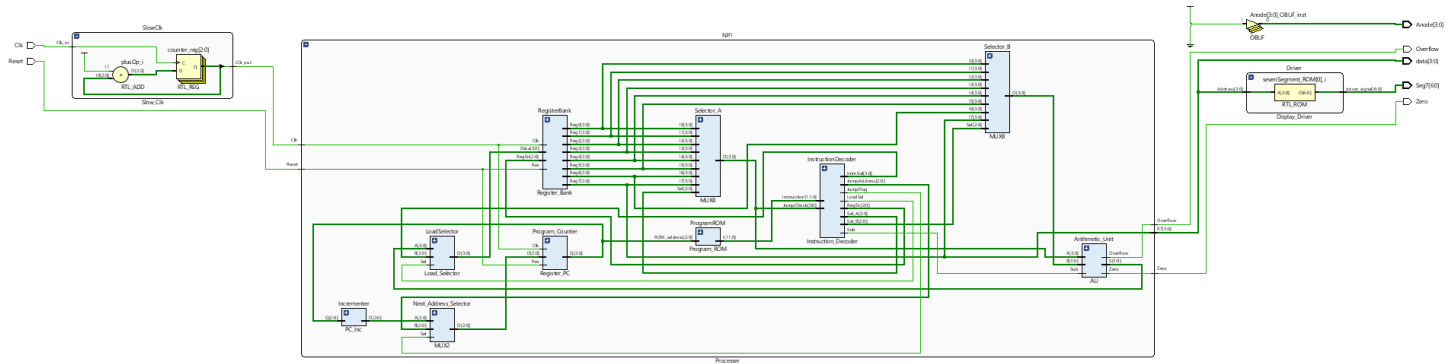
Design objectives emphasise resource efficiency (minimising Slice LUT and Register usage), clear modular hierarchy for reuse and verification, and demonstrable operation on an FPGA dev board with human-readable output through a 7-segment display LUT.

3. Assembly Program & Machine Code

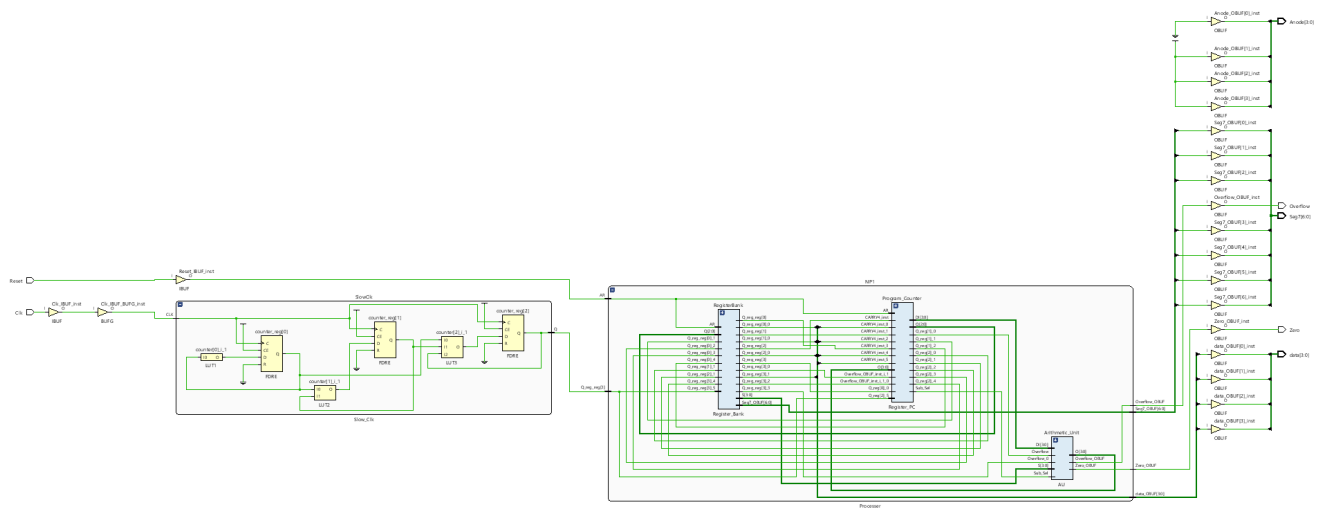
The following table maps assembly mnemonics to their 12-bit machine code representation used in the Program ROM.

Assembly Code	Machine Code (12-bit)
MOVI R7, 3	101110000011
MOVI R1, 1	100010000001
NEG R1	010010000000
MOVI R2, 3	100100000011
ADD R2, R1	000100010000
ADD R7, R2	001110100000
JZR R2, 6	110100000110
JZR R0, 4	110000000100

4. Nano Processor Design Schematic



RTL Analysis : schematic



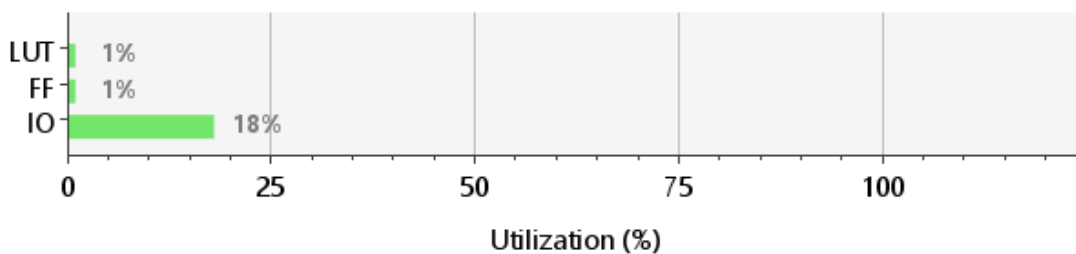
Synthesis : schematic

5. Synthesis Report Summary

Name ¹	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	Bonded IOB (106)	BUFGCTRL (32)
▼ N NanoProcessor	24	18	1	19	1
> I NP1 (Processer)	22	15	1	0	0
I SlowClk (Slow_Clk)	2	3	0	0	0

Summary

Resource	Utilization	Available	Utilization %
LUT	24	20800	0.12
FF	18	41600	0.04
IO	19	106	17.92



6. Slow_Clk

6.1. VHDL Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Slow_Clk is
    generic (
        N : INTEGER := 2
    );
    Port (
        Clk_in  : in  STD_LOGIC;
        Clk_out : out STD_LOGIC
    );
end Slow_Clk;

architecture Behavioral of Slow_Clk is

    signal counter : unsigned(N downto 0) := (others => '0'); -- 27-bit binary
counter

begin
    process(Clk_in)
    begin
        if rising_edge(Clk_in) then
            counter <= counter + 1;
        end if;
    end process;

    Clk_out <= counter(N);

end Behavioral;
```

6.2. Testbench

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity TB_Slow_Clk is
end entity TB_Slow_Clk;

architecture Behavioral of TB_Slow_Clk is

    component Slow_Clk
        generic (
            N : INTEGER := 2
        );
        port (
            clk_in  : in  std_logic;
            clk_out  : out std_logic
        );
    end component;

    signal clk_in  : std_logic := '0';
    signal clk_out : std_logic;

    constant clk_period : time := 20 ns;

begin

    uut: Slow_Clk
        generic map (
            N => 2
        )
        port map (
            clk_in  => clk_in,
            clk_out => clk_out
        );

    clk_process : process
    begin
        while true loop
            clk_in <= '0';
            wait for clk_period / 2;
            clk_in <= '1';
            wait for clk_period / 2;
        end loop
    end process
end architecture Behavioral;
```

```

        end loop;
    end process;

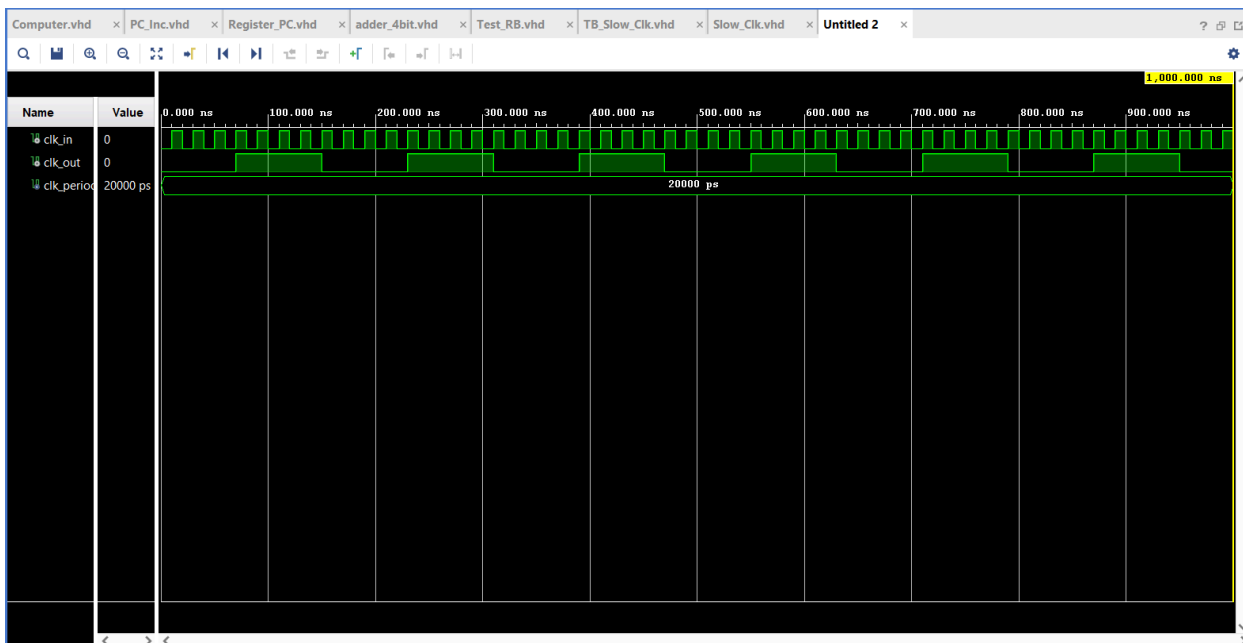
    stim_process : process
    begin
        wait for 100 ns;
        wait for 1000 ns;
        wait for 40 ns;
        wait for 1000 ns;
        wait;
    end process;

end architecture Behavioral;

```

6.3. Timing Diagram

This timing diagram shows the relationship between the clock input, reset signal, and the slow clock output waveform. It illustrates how the slow clock responds over time as the clock and reset signals change.



7. 4bit Adder

7.1. VHDL Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

library UNISIM;
use UNISIM.VComponents.all;

entity Adder_4bit is
    Port ( A : in  STD_LOGIC_VECTOR(3 downto 0);
          B : in  STD_LOGIC_VECTOR(3 downto 0);
          C_in : in  STD_LOGIC;
          S : out STD_LOGIC_VECTOR(3 downto 0);
          C_out : out STD_LOGIC);
end Adder_4bit;

architecture Structural of Adder_4bit is

    signal P : STD_LOGIC_VECTOR(3 downto 0);
    signal CO : STD_LOGIC_VECTOR(3 downto 0);

begin

    P(0) <= A(0) XOR B(0);
    P(1) <= A(1) XOR B(1);
    P(2) <= A(2) XOR B(2);
    P(3) <= A(3) XOR B(3);

    CARRY4_inst : CARRY4
        port map (
            CI => C_in,
            CYINIT => '0',
            DI => A,
            S => P,
            O => S,
            CO => CO
        );

    C_out <= CO(3);

end Structural;
```

7.2. Testbench

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity TB_adder_4bit is
end TB_adder_4bit;

architecture Behavioral of TB_adder_4bit is

    component Adder_4bit is
        Port ( A      : in  STD_LOGIC_VECTOR(3 downto 0);
              B      : in  STD_LOGIC_VECTOR(3 downto 0);
              C_in   : in  STD_LOGIC;
              S      : out STD_LOGIC_VECTOR(3 downto 0);
              C_out  : out STD_LOGIC);
    end component;

    signal A, B : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
    signal C_in : STD_LOGIC := '0';
    signal S : STD_LOGIC_VECTOR(3 downto 0);
    signal C_out : STD_LOGIC;

begin

    UUT : adder_4bit
        port map (A => A, B => B, C_in => C_in, S => S, C_out => C_out);

    process
    begin
        -- Test 1: 0 + 0 = 0
        A <= "0000"; B <= "0000"; C_in <= '0'; wait for 50 ns;
        -- Test 2: 1 + 1 = 2
        A <= "0001"; B <= "0001"; C_in <= '0'; wait for 50 ns;
        -- Test 3: 5 + 3 = 8
        A <= "0101"; B <= "0011"; C_in <= '0'; wait for 50 ns;
        -- Test 4: 7 + 7 = 14
        A <= "0111"; B <= "0111"; C_in <= '0'; wait for 50 ns;
        -- Test 5: 15 + 1 = 16, sum=0 carry=1
        A <= "1111"; B <= "0001"; C_in <= '0'; wait for 50 ns;
        -- Test 6: 15 + 15 = 30, sum=14 carry=1
        A <= "1111"; B <= "1111"; C_in <= '0'; wait for 50 ns;
        -- Test 7: 7 + 7 + cin=1 = 15
    end process;

```

```

A <= "0111"; B <= "0111"; C_in <= '1'; wait for 50 ns;
-- Test 8: 15 + 0 + cin=1 = 16, sum=0 carry=1
A <= "1111"; B <= "0000"; C_in <= '1'; wait for 50 ns;
-- Test 9: 3 + 12 = 15
A <= "0011"; B <= "1100"; C_in <= '0'; wait for 50 ns;
-- Test 10: 8 + 8 = 16, sum=0 carry=1
A <= "1000"; B <= "1000"; C_in <= '0'; wait for 50 ns;

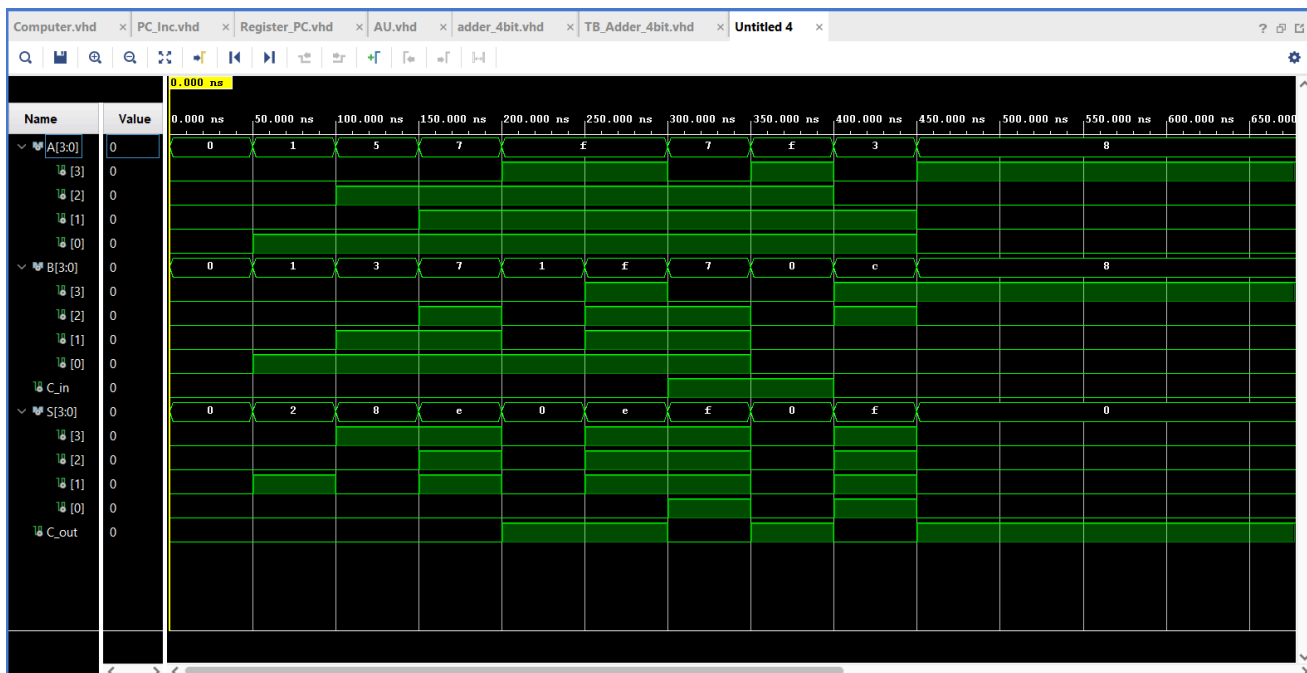
    wait;
end process;

end Behavioral;

```

7.3. Timing Diagram

This timing diagram shows the operational behaviour of the Adder_4bit component, showcasing the dynamic interactions between its inputs and outputs. It specifically details the relationship between input A, input B, the carry-in signal, the sum output, and the carry-out signal. The diagram visually represents how these signals change over time, demonstrating the component's 4-bit addition across various test cases, including zero addition, mid-range values, maximum values, and carry propagation scenarios.



8. Arithmetic Unit

8.1. VHDL Code

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity AU is
    Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
          B : in STD_LOGIC_VECTOR (3 downto 0);
          Sub : in STD_LOGIC;
          S : out STD_LOGIC_VECTOR (3 downto 0);
          Overflow : out STD_LOGIC;
          Zero : out STD_LOGIC);
end AU;

architecture Behavioral of AU is
    Component Adder_4bit is
        Port (      A : in STD_LOGIC_VECTOR (3 downto 0);
                  B : in STD_LOGIC_VECTOR (3 downto 0);
                  C_in : in STD_LOGIC;
                  S : out STD_LOGIC_VECTOR (3 downto 0);
                  C_out : out STD_LOGIC);
    end Component;

    Signal Tmp_B : STD_LOGIC_VECTOR (3 downto 0);
    Signal Sum : STD_LOGIC_VECTOR (3 downto 0);
    Signal C_out_sig : STD_LOGIC;

begin
    RCA : Adder_4bit
        port map(
            A => A,
            B => Tmp_B,
            C_in => Sub,
            S => Sum,
            C_out => C_out_sig
        );

    Tmp_B <= B XOR (Sub & Sub & Sub & Sub);
    S <= Sum;

    Zero <= '1' when Sum = "0000" else '0';

```

```

Overflow <= (NOT A(3) AND NOT Tmp_B(3) AND Sum(3))
           OR (A(3) AND Tmp_B(3) AND NOT Sum(3));

end Behavioral;

```

8.2. Testbench

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_AU is
end TB_AU;

architecture Behavioral of TB_AU is

    signal A, B : STD_LOGIC_VECTOR(3 downto 0);
    signal Sub : STD_LOGIC;
    signal S : STD_LOGIC_VECTOR(3 downto 0);
    signal Overflow : STD_LOGIC;
    signal Zero : STD_LOGIC;

    component AU is
        Port ( A : in STD_LOGIC_VECTOR(3 downto 0);
              B : in STD_LOGIC_VECTOR(3 downto 0);
              Sub : in STD_LOGIC;
              S : out STD_LOGIC_VECTOR(3 downto 0);
              Overflow : out STD_LOGIC;
              Zero : out STD_LOGIC);
    end component;

begin

    UUT: AU
        port map (
            A => A,
            B => B,
            Sub => Sub,
            S => S,
            Overflow => Overflow,
            Zero => Zero
        );

```

```

process
begin
    -- Test 1: 1 + 2 = 3
    A <= "0001"; B <= "0010"; Sub <= '0'; wait for 50 ns;

    -- Test 2: 4 - 3 = 1
    A <= "0100"; B <= "0011"; Sub <= '1'; wait for 50 ns;

    -- Test 3: 3 - 3 = 0 (zero flag)
    A <= "0011"; B <= "0011"; Sub <= '1'; wait for 50 ns;

    -- Test 4: 7 + 1 = 8 (overflow)
    A <= "0111"; B <= "0001"; Sub <= '0'; wait for 50 ns;

    -- Test 5: 15 + 1 = 16, wraps to 0 (overflow + zero)
    A <= "1111"; B <= "0001"; Sub <= '0'; wait for 50 ns;

    -- Test 6: 8 - 7 = 1
    A <= "1000"; B <= "0111"; Sub <= '1'; wait for 50 ns;

    -- Test 7: 0 + 0 = 0 (zero flag)
    A <= "0000"; B <= "0000"; Sub <= '0'; wait for 50 ns;

    -- Test 8: 5 - 5 = 0 (zero flag)
    A <= "0101"; B <= "0101"; Sub <= '1'; wait for 50 ns;

    wait;
end process;

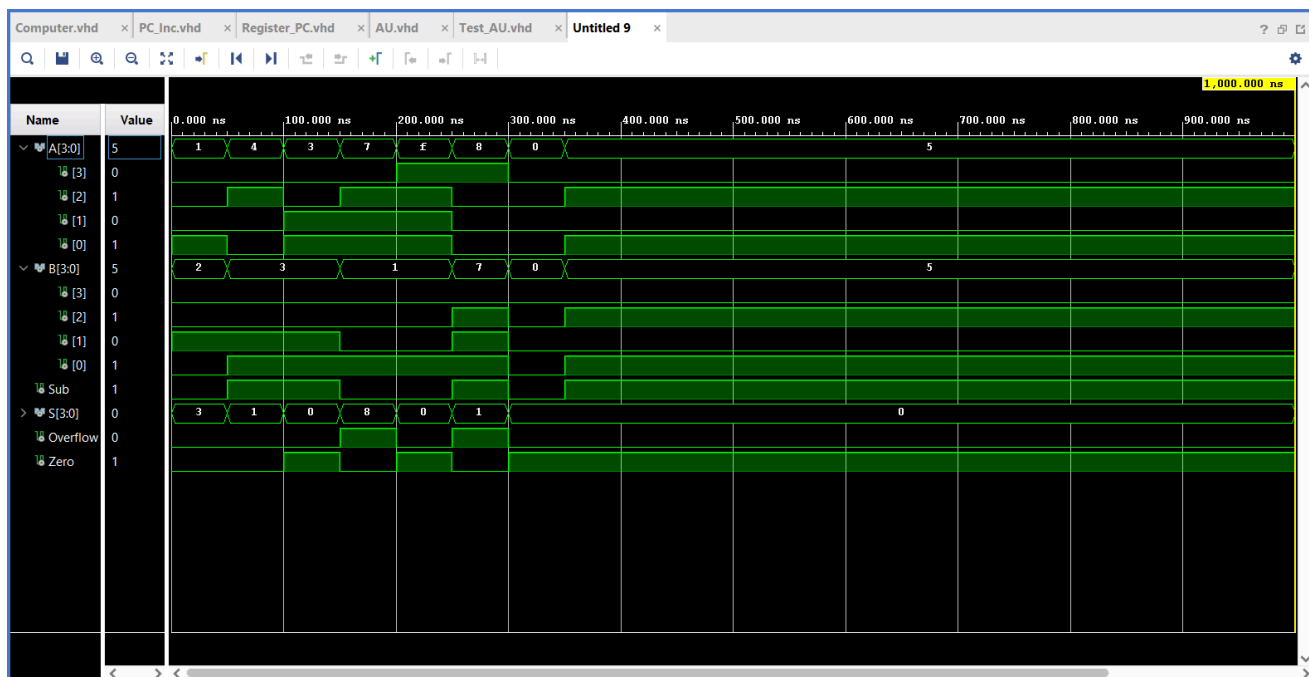
end Behavioral;

```

8.3. Timing Diagram

This section provides a timing diagram description for the AU component, illustrating the interactions and timing relationships between its key signals: the data inputs (A and B), the subtract enable signal (Sub), the result output (S), the overflow flag (Overflow), and the zero flag (Zero).

It highlights how the output changes in response to different input combinations, how the Sub signal switches the unit between addition and subtraction mode, and how the flags reflect the arithmetic result.



The diagram visually represents the combinational behaviour of the 4-bit arithmetic unit. It clearly demonstrates how Overflow is asserted when the result of a signed operation exceeds the representable range, detected by comparing the sign bits of the inputs and the result. Furthermore, it illustrates how the Zero flag is asserted whenever the result S is exactly zero, regardless of the operation mode. When Sub is low, the unit performs addition; when high, it performs two's complement subtraction by inverting B and adding one through the carry-in.

9. 4bit Register

9.1. VHDL Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Register_4bit is
    Port (
        Clk      : in STD_LOGIC;
        Res      : in STD_LOGIC;
        En       : in STD_LOGIC;
        D        : in STD_LOGIC_VECTOR(3 downto 0);
        Q        : out STD_LOGIC_VECTOR(3 downto 0)
    );
end Register_4bit;

architecture Behavioral of Register_4bit is
    signal Q_reg : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
begin
    process(Clk)
    begin
        if rising_edge(Clk) then
            if Res = '0' then
                if En = '1' then
                    Q_reg <= D;
                end if;
            else
                Q_reg <= "0000";
            end if;
        end if;
    end process;

    Q <= Q_reg;
end Behavioral;
```

9.2. Testbench

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Register_4bit is
    Port (
        Clk      : in STD_LOGIC;
        Res      : in STD_LOGIC;
        En       : in STD_LOGIC;
        D        : in STD_LOGIC_VECTOR(3 downto 0);
        Q        : out STD_LOGIC_VECTOR(3 downto 0)
    );
end Register_4bit;

architecture Behavioral of Register_4bit is
    signal Q_reg : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
begin
    process(Clk)
    begin
        if rising_edge(Clk) then
            if Res = '0' then
                if En = '1' then
                    Q_reg <= D;
                end if;
            else
                Q_reg <= "0000";
            end if;
        end if;
    end process;

    Q <= Q_reg;
end Behavioral;

signal Clk : STD_LOGIC := '0';
signal Res : STD_LOGIC := '0';
signal En  : STD_LOGIC := '0';
signal D   : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
signal Q   : STD_LOGIC_VECTOR(3 downto 0);

constant CLK_PERIOD : time := 20 ns;
begin
    UUT : Register_4bit
        port map (Clk => Clk, Res => Res, En => En, D => D, Q => Q);

    Clk <= not Clk after CLK_PERIOD / 2;

```

```

process
begin
    -- Test 1: Synchronous reset clears output
    Res <= '1'; En <= '0'; D <= "1010";
    wait for CLK_PERIOD;

    -- Test 2: Data NOT captured when En=0
    Res <= '0'; En <= '0'; D <= "1010";
    wait for CLK_PERIOD;

    -- Test 3: Enable=1 -> data captured on rising edge
    En <= '1'; D <= "1010";
    wait for CLK_PERIOD;

    -- Test 4: Different data value
    D <= "0101";
    wait for CLK_PERIOD;

    -- Test 5: En=0 -> data held
    En <= '0'; D <= "1111";
    wait for CLK_PERIOD;

    -- Test 6: Synchronous reset overrides enable
    En <= '1'; Res <= '1'; D <= "1111";
    wait for CLK_PERIOD;

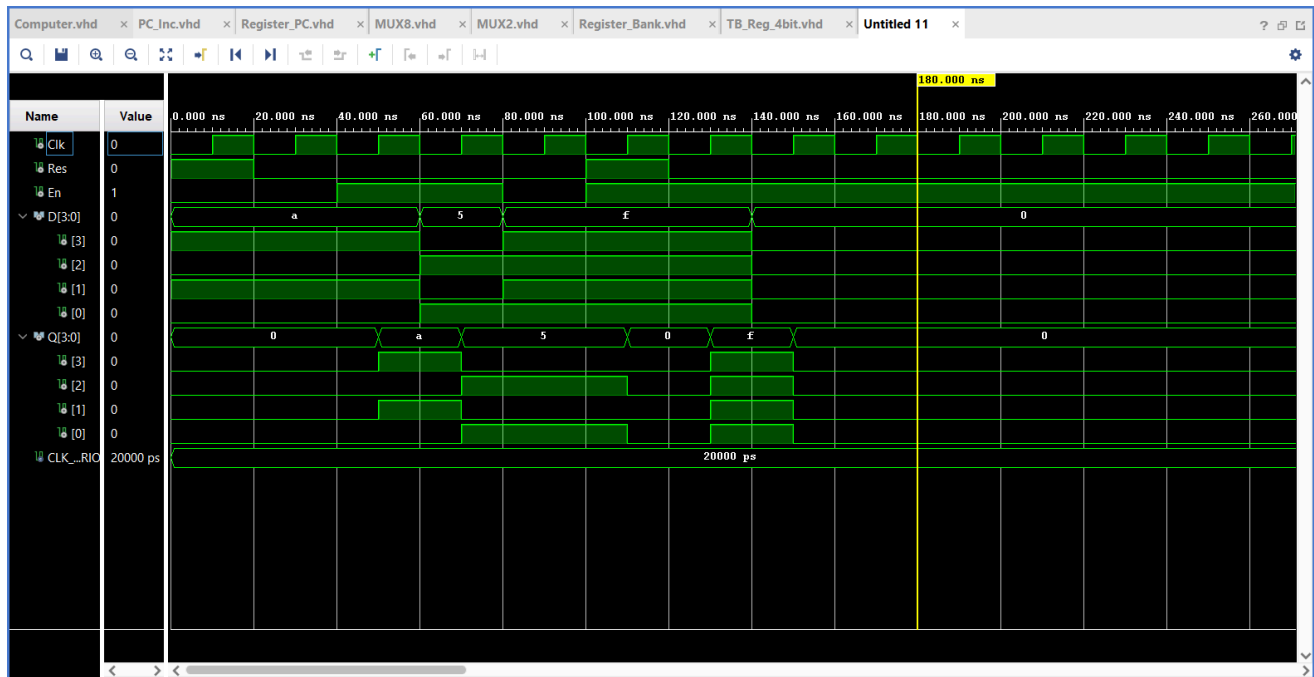
    -- Test 7: Load 0xF
    Res <= '0'; En <= '1'; D <= "1111";
    wait for CLK_PERIOD;

    -- Test 8: Load 0x0
    D <= "0000";
    wait for CLK_PERIOD;

    wait;
end process;
end Behavioral;

```

9.3. Timing Diagram



10. Register Bank

10.1. VHDL Code

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Register_4bit is
    Port (
        Clk      : in STD_LOGIC;
        Res      : in STD_LOGIC;
        En       : in STD_LOGIC;
        D        : in STD_LOGIC_VECTOR(3 downto 0);
        Q        : out STD_LOGIC_VECTOR(3 downto 0)
    );
end Register_4bit;

architecture Behavioral of Register_4bit is
    signal Q_reg : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
begin
    process(Clk)
    begin
        if rising_edge(Clk) then
            if Res = '0' then
                if En = '1' then
                    Q_reg <= D;
                end if;
            else
                Q_reg <= "0000";
            end if;
        end if;
    end process;

    Q <= Q_reg;
end Behavioral;

signal Clk : STD_LOGIC := '0';
signal Res : STD_LOGIC := '0';
signal En  : STD_LOGIC := '0';
signal D   : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
signal Q   : STD_LOGIC_VECTOR(3 downto 0);

constant CLK_PERIOD : time := 20 ns;

```

```

begin
  UUT : Register_4bit
    port map (Clk => Clk, Res => Res, En => En, D => D, Q => Q);

  Clk <= not Clk after CLK_PERIOD / 2;

  process
  begin
    -- Test 1: Synchronous reset clears output
    Res <= '1'; En <= '0'; D <= "1010";
    wait for CLK_PERIOD;

    -- Test 2: Data NOT captured when En=0
    Res <= '0'; En <= '0'; D <= "1010";
    wait for CLK_PERIOD;

    -- Test 3: Enable=1 -> data captured on rising edge
    En <= '1'; D <= "1010";
    wait for CLK_PERIOD;

    -- Test 4: Different data value
    D <= "0101";
    wait for CLK_PERIOD;

    -- Test 5: En=0 -> data held
    En <= '0'; D <= "1111";
    wait for CLK_PERIOD;

    -- Test 6: Synchronous reset overrides enable
    En <= '1'; Res <= '1'; D <= "1111";
    wait for CLK_PERIOD;

    -- Test 7: Load 0xF
    Res <= '0'; En <= '1'; D <= "1111";
    wait for CLK_PERIOD;

    -- Test 8: Load 0x0
    D <= "0000";
    wait for CLK_PERIOD;

    wait;
  end process;
end Behavioral; TD_LOGIC_VECTOR(3 DOWNTO 0);
Reg6      : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
Reg7      : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)

```

```

);
END Register_Bank;

ARCHITECTURE Behavioral OF Register_Bank IS

    COMPONENT Register_4bit
    PORT (
        Clk    : IN  STD_LOGIC;

        Res    : IN  STD_LOGIC;
        En     : IN  STD_LOGIC;
        D      : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
        Q      : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
    );

    END COMPONENT;

    COMPONENT Decoder_8
    PORT (
        Sel : in  STD_LOGIC_VECTOR(2 downto 0);
        O   : out STD_LOGIC_VECTOR(7 downto 0)
    );
    END COMPONENT;

    SIGNAL Reg_Sel : STD_LOGIC_VECTOR(7 DOWNTO 0);

BEGIN

    Decoder_3_to_8 : Decoder_8
    PORT MAP (
        Sel => RegEn,
        O   => Reg_Sel
    );

    reg_0 : Register_4bit
    PORT MAP (
        Clk    => Clk,
        Res    => Res,
        En     => '1',
        D      => "0000",
        Q      => Reg0
    );

    reg_1 : Register_4bit
    PORT MAP (
        Clk    => Clk,

```

```

        Res    => Res,
        En     => Reg_Sel(1),
        D      => Data,
        Q      => Reg1
    );

reg_2 : Register_4bit
    PORT MAP (
        Clk    => Clk,
        Res    => Res,
        En     => Reg_Sel(2),
        D      => Data,
        Q      => Reg2
    );

reg_3 : Register_4bit
    PORT MAP (
        Clk    => Clk,
        Res    => Res,
        En     => Reg_Sel(3),
        D      => Data,
        Q      => Reg3
    );

reg_4 : Register_4bit
    PORT MAP (
        Clk    => Clk,
        Res    => Res,
        En     => Reg_Sel(4),
        D      => Data,
        Q      => Reg4
    );

reg_5 : Register_4bit
    PORT MAP (
        Clk    => Clk,
        Res    => Res,
        En     => Reg_Sel(5),
        D      => Data,
        Q      => Reg5
    );

reg_6 : Register_4bit
    PORT MAP (
        Clk    => Clk,

```

```

        Res    => Res,
        En     => Reg_Sel(6),
        D      => Data,
        Q      => Reg6
    );

    reg_7 : Register_4bit
        PORT MAP (
            Clk   => Clk,
            Res   => Res,
            En    => Reg_Sel(7),
            D     => Data,
            Q     => Reg7
        );

END Behavioral;

```

10.2. Testbench

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_Reg_Bank is
end TB_Reg_Bank;

architecture Behavioral of TB_Reg_Bank is

    component Register_Bank is
        Port ( RegEn : in  STD_LOGIC_VECTOR(2 downto 0);
              Data  : in  STD_LOGIC_VECTOR(3 downto 0);
              Clk   : in  STD_LOGIC;
              Res   : in  STD_LOGIC;
              Reg0  : out STD_LOGIC_VECTOR(3 downto 0);
              Reg1  : out STD_LOGIC_VECTOR(3 downto 0);
              Reg2  : out STD_LOGIC_VECTOR(3 downto 0);
              Reg3  : out STD_LOGIC_VECTOR(3 downto 0);
              Reg4  : out STD_LOGIC_VECTOR(3 downto 0);
              Reg5  : out STD_LOGIC_VECTOR(3 downto 0);
              Reg6  : out STD_LOGIC_VECTOR(3 downto 0);

```

```

        Reg7 : out STD_LOGIC_VECTOR(3 downto 0));
end component;

signal Clk : STD_LOGIC := '0';
signal Res : STD_LOGIC := '1';
signal RegEn: STD_LOGIC_VECTOR(2 downto 0) := "000";
signal Data : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
signal Reg0,Reg1,Reg2,Reg3 : STD_LOGIC_VECTOR(3 downto 0);
signal Reg4,Reg5,Reg6,Reg7 : STD_LOGIC_VECTOR(3 downto 0);

constant CLK_PERIOD : time := 20 ns;

begin

    UUT : Register_Bank
        port map (RegEn=>RegEn, Data=>Data, Clk=>Clk, Res=>Res,
            Reg0=>Reg0, Reg1=>Reg1, Reg2=>Reg2, Reg3=>Reg3,
            Reg4=>Reg4, Reg5=>Reg5, Reg6=>Reg6, Reg7=>Reg7);

    Clk <= not Clk after CLK_PERIOD / 2;

    stim : process
    begin
        -- Reset: all registers should be 0
        Res <= '1'; wait for CLK_PERIOD;

        Res <= '0'; wait for 5 ns;

        -- R0 is permanently zero - attempt to write
        RegEn <= "000"; Data <= "1111"; wait for CLK_PERIOD;

        -- Write R1 = 1
        RegEn <= "001"; Data <= "0001"; wait for CLK_PERIOD;

        -- Write R2 = 2
        RegEn <= "010"; Data <= "0010"; wait for CLK_PERIOD;

        -- Write R3 = 3
        RegEn <= "011"; Data <= "0011"; wait for CLK_PERIOD;

        -- Write R4 = 4
        RegEn <= "100"; Data <= "0100"; wait for CLK_PERIOD;

        -- Write R5 = 5
        RegEn <= "101"; Data <= "0101"; wait for CLK_PERIOD;
    end process;

```

```

-- Write R6 = 6
RegEn <= "110"; Data <= "0110"; wait for CLK_PERIOD;

-- Write R7 = 7
RegEn <= "111"; Data <= "0111"; wait for CLK_PERIOD;

-- Overwrite R7 = 15
RegEn <= "111"; Data <= "1111"; wait for CLK_PERIOD;

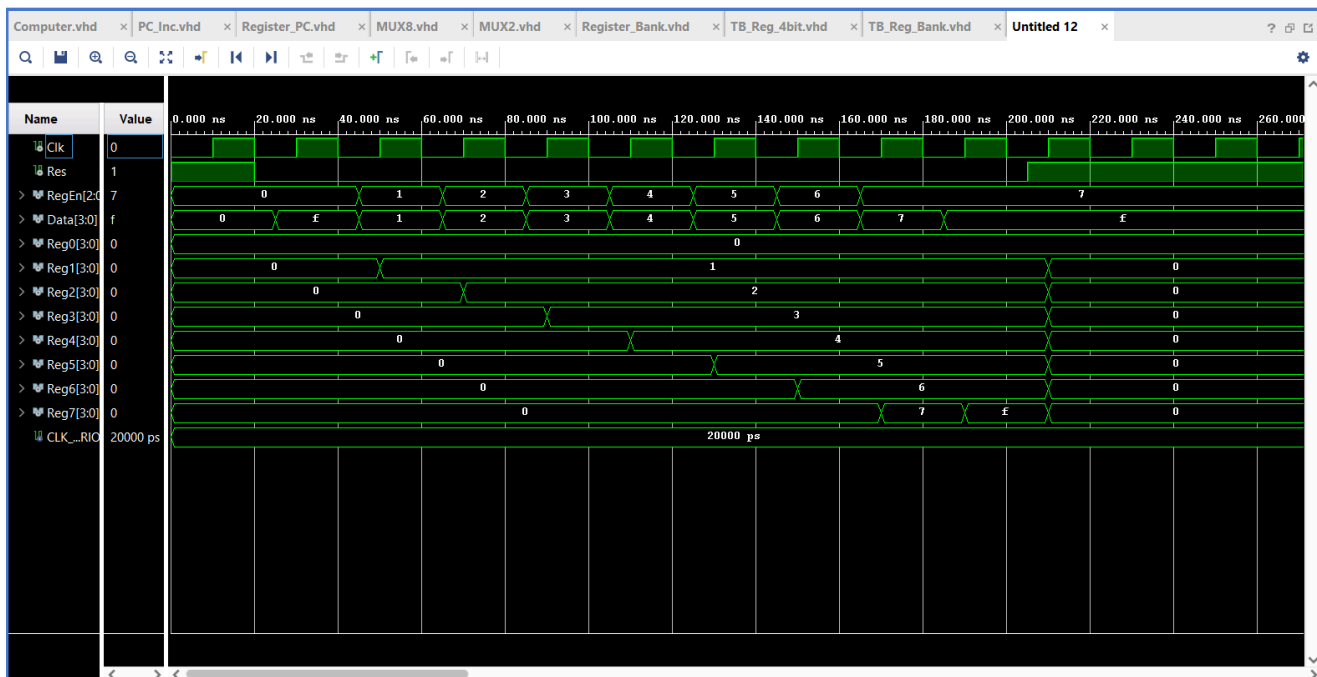
-- Reset again
Res <= '1'; wait for CLK_PERIOD;

    wait;
end process;

end Behavioral;

```

10.3. Timing Diagram



11. 2-to-1 Multiplexer

11.1. VHDL Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX2 is
    Port ( A : in  STD_LOGIC_VECTOR(2 downto 0);
          B : in  STD_LOGIC_VECTOR(2 downto 0);
          Sel : in  STD_LOGIC;
          O : out STD_LOGIC_VECTOR(2 downto 0));
end MUX2;

architecture Behavioral of MUX2 is

begin
    O <= A when Sel = '0' else B;
end behavioral;
```

11.2. Testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_MUX2 is
end TB_MUX2;

architecture Behavioral of TB_MUX2 is

    component MUX2 is
        Port ( A : in  STD_LOGIC_VECTOR(2 downto 0);
              B : in  STD_LOGIC_VECTOR(2 downto 0);
              Sel : in  STD_LOGIC;
              O : out STD_LOGIC_VECTOR(2 downto 0));
    end component;

    signal A, B : STD_LOGIC_VECTOR(2 downto 0) := "000";
    signal Sel : STD_LOGIC := '0';
```



```
signal O      : STD_LOGIC_VECTOR(2 downto 0);

begin

  UUT : MUX2 port map (A => A, B => B, Sel => Sel, O => O);

  process
  begin
    -- Test 1: Sel=0 passes A
    A <= "001"; B <= "110"; Sel <= '0'; wait for 100 ns;

    -- Test 2: Sel=1 passes B
    Sel <= '1'; wait for 100 ns;

    -- Test 3: PC sequential address (Sel=0)
    A <= "011"; B <= "101"; Sel <= '0'; wait for 100 ns;

    -- Test 4: Jump address (Sel=1)
    Sel <= '1'; wait for 100 ns;

    -- Test 5: Both inputs same
    A <= "100"; B <= "100"; Sel <= '0'; wait for 100 ns;
    Sel <= '1'; wait for 100 ns;

    -- Test 6: All zeros vs all ones
    A <= "000"; B <= "111"; Sel <= '0'; wait for 100 ns;
    Sel <= '1'; wait for 100 ns;

    wait;
  end process;

end Behavioral;
```

11.3. Timing Diagram



12. 8-to-1 Multiplexer

12.1. VHDL Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity MUX8 is
    Port (
        Sel : in  STD_LOGIC_VECTOR(2 downto 0);

    I0  : in  STD_LOGIC_VECTOR(3 downto 0);
    I1  : in  STD_LOGIC_VECTOR(3 downto 0);
    I2  : in  STD_LOGIC_VECTOR(3 downto 0);
    I3  : in  STD_LOGIC_VECTOR(3 downto 0);
    I4  : in  STD_LOGIC_VECTOR(3 downto 0);
    I5  : in  STD_LOGIC_VECTOR(3 downto 0);
    I6  : in  STD_LOGIC_VECTOR(3 downto 0);
    I7  : in  STD_LOGIC_VECTOR(3 downto 0);
    O   : out STD_LOGIC_VECTOR(3 downto 0));
end MUX8;
architecture Behavioral of MUX8 is
begin
    process(Sel, I0, I1, I2, I3, I4, I5, I6, I7)
    begin
        case Sel is
            when "000" => O <= I0;
            when "001" => O <= I1;
            when "010" => O <= I2;
            when "011" => O <= I3;
            when "100" => O <= I4;
            when "101" => O <= I5;
            when "110" => O <= I6;
            when "111" => O <= I7;
            when others => O <= (others => '0');
        end case;
    end process;
end Behavioral;
```

12.2. Testbench

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_MUX8 is
end TB_MUX8;

architecture Behavioral of TB_MUX8 is

    component MUX8 is
        Port ( Sel : in  STD_LOGIC_VECTOR(2 downto 0);
              I0  : in  STD_LOGIC_VECTOR(3 downto 0);
              I1  : in  STD_LOGIC_VECTOR(3 downto 0);
              I2  : in  STD_LOGIC_VECTOR(3 downto 0);
              I3  : in  STD_LOGIC_VECTOR(3 downto 0);
              I4  : in  STD_LOGIC_VECTOR(3 downto 0);
              I5  : in  STD_LOGIC_VECTOR(3 downto 0);
              I6  : in  STD_LOGIC_VECTOR(3 downto 0);
              I7  : in  STD_LOGIC_VECTOR(3 downto 0);
              O   : out STD_LOGIC_VECTOR(3 downto 0));
    end component;

    signal Sel       : STD_LOGIC_VECTOR(2 downto 0) := "000";
    signal I0,I1,I2,I3 : STD_LOGIC_VECTOR(3 downto 0);
    signal I4,I5,I6,I7 : STD_LOGIC_VECTOR(3 downto 0);
    signal O         : STD_LOGIC_VECTOR(3 downto 0);

begin

    I0 <= "0000";
    I1 <= "0001";
    I2 <= "0010";
    I3 <= "0011";
    I4 <= "0100";
    I5 <= "0101";
    I6 <= "0110";
    I7 <= "0111";

    UUT : MUX8
        port map (
            Sel=>Sel, I0=>I0, I1=>I1,
            I2=>I2, I3=>I3, I4=>I4,
            I5=>I5, I6=>I6, I7=>I7, O=>O
        );
end architecture Behavioral of TB_MUX8;

```

```

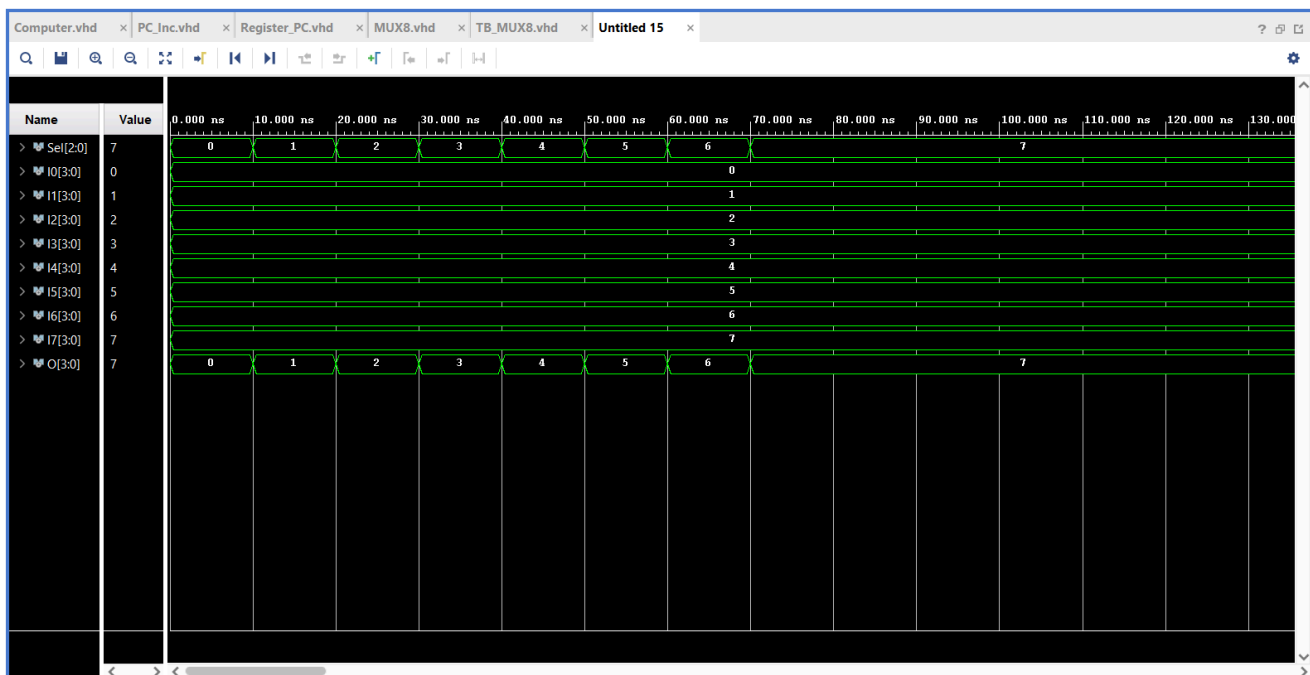
);

process
begin
    Sel <= "000"; wait for 10 ns;
    Sel <= "001"; wait for 10 ns;
    Sel <= "010"; wait for 10 ns;
    Sel <= "011"; wait for 10 ns;
    Sel <= "100"; wait for 10 ns;
    Sel <= "101"; wait for 10 ns;
    Sel <= "110"; wait for 10 ns;
    Sel <= "111"; wait for 10 ns;
    wait;
end process;

end Behavioral;

```

12.3. Timing Diagram



13. 3-to-8 Decoder

13.1. VHDL Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Decoder_8 is
    Port (
        Sel : in  STD_LOGIC_VECTOR(2 downto 0);
        O    : out STD_LOGIC_VECTOR(7 downto 0)
    );
end Decoder_8;

architecture Behavioral of Decoder_8 is
begin
    process(Sel)
    begin
        case Sel is
            when "000" => O <= "00000001";
            when "001" => O <= "00000010";
            when "010" => O <= "00000100";
            when "011" => O <= "00001000";
            when "100" => O <= "00010000";
            when "101" => O <= "00100000";
            when "110" => O <= "01000000";
            when "111" => O <= "10000000";
            when others => O <= (others => '0');
        end case;
    end process;
end Behavioral;
```

13.2. Testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_3to8_Dec is
end TB_3to8_Dec;

architecture Behavioral of TB_3to8_Dec is

    component Decoder_8 is
        Port ( Sel : in  STD_LOGIC_VECTOR(2 downto 0);
              O   : out STD_LOGIC_VECTOR(7 downto 0));
    end component;

    signal Sel : STD_LOGIC_VECTOR(2 downto 0) := "000";
    signal O   : STD_LOGIC_VECTOR(7 downto 0);

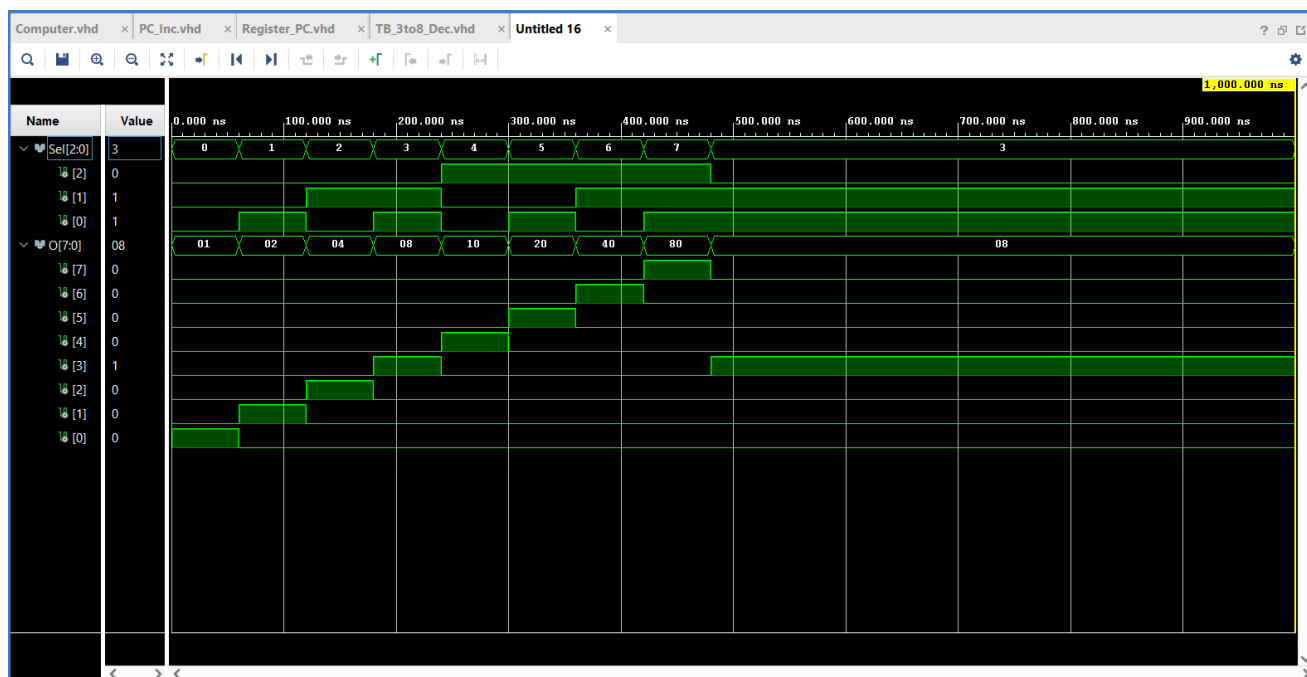
begin

    UUT : Decoder_8 port map (Sel => Sel, O => O);

    process
    begin
        Sel <= "000"; wait for 60 ns;
        Sel <= "001"; wait for 60 ns;
        Sel <= "010"; wait for 60 ns;
        Sel <= "011"; wait for 60 ns;
        Sel <= "100"; wait for 60 ns;
        Sel <= "101"; wait for 60 ns;
        Sel <= "110"; wait for 60 ns;
        Sel <= "111"; wait for 60 ns;
        Sel <= "011"; wait for 60 ns;
        wait;
    end process;

end Behavioral;
```

13.3. Timing Diagram



14. Program ROM

14.1. VHDL Code

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Program_ROM is
    port( ROM_address : in  STD_LOGIC_VECTOR(2 downto 0);
          I           : out STD_LOGIC_VECTOR(11 downto 0));
end Program_ROM;

architecture Behavioral of Program_ROM is

    type ROM_Type is array (0 to 7) of STD_LOGIC_VECTOR(11 downto 0);
    constant Program : ROM_Type := (
        "101110000011", -- MOVI R7, 3
        "100010000001", -- MOVI R1, 1
        "010010000000", -- NEG  R1
        "100100000011", -- MOVI R2, 3
        "000100010000", -- ADD  R2, R1
        "001110100000", -- ADD  R7, R2
        "110100000110", -- JZR  R2, 6
        "110000000100"  -- JZR  R0, 4
    );

begin
    I <= Program(to_integer(unsigned(ROM_address)));
end Behavioral;

```

14.2. Testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_Program_ROM is
end TB_Program_ROM;

architecture Behavioral of TB_Program_ROM is

    component Program_ROM is
        Port ( ROM_address : in  STD_LOGIC_VECTOR(2 downto 0);
              I           : out STD_LOGIC_VECTOR(11 downto 0));
    end component;

    signal ROM_address : STD_LOGIC_VECTOR(2 downto 0) := "000";
    signal I           : STD_LOGIC_VECTOR(11 downto 0);

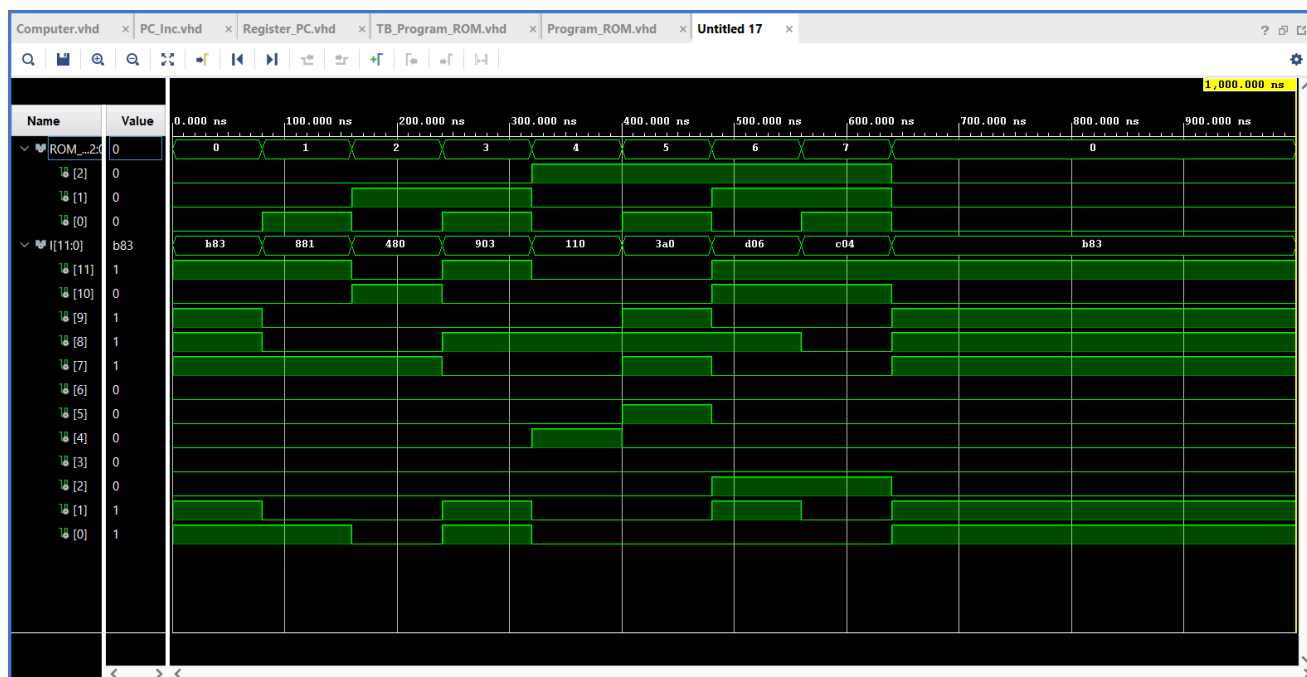
begin

    UUT : Program_ROM port map (ROM_address => ROM_address, I => I);

    process
    begin
        ROM_address <= "000"; wait for 80 ns;
        ROM_address <= "001"; wait for 80 ns;
        ROM_address <= "010"; wait for 80 ns;
        ROM_address <= "011"; wait for 80 ns;
        ROM_address <= "100"; wait for 80 ns;
        ROM_address <= "101"; wait for 80 ns;
        ROM_address <= "110"; wait for 80 ns;
        ROM_address <= "111"; wait for 80 ns;
        ROM_address <= "000"; wait for 80 ns;
        wait;
    end process;

end Behavioral;
```

14.3. Timing Diagram



15. Program Counter

15.1. VHDL Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Register_PC is
  Port (
    Clk  : in  STD_LOGIC;           -- Clock input
    Res  : in  STD_LOGIC;           -- Reset input (active high)
    D    : in  STD_LOGIC_VECTOR(2 downto 0); -- 3-bit data input
    Q    : out STD_LOGIC_VECTOR(2 downto 0) -- 3-bit data output
  );
end Register_PC;

architecture Behavioral of Register_PC is
begin
  process(Clk, Res)
  begin
    if Res = '1' then
      Q <= (others => '0');          -- async reset: fires immediately
    elsif rising_edge(Clk) then
      Q <= D;
    end if;
  end process;
end Behavioral;
```

15.2. Testbench

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_PC is
end TB_PC;

architecture Behavioral of TB_PC is

    component Register_PC is
        Port ( Clk : in  STD_LOGIC;
              Res : in  STD_LOGIC;
              D   : in  STD_LOGIC_VECTOR(2 downto 0);
              Q   : out STD_LOGIC_VECTOR(2 downto 0));
    end component;

    signal Clk : STD_LOGIC := '0';
    signal Res : STD_LOGIC := '0';
    signal D   : STD_LOGIC_VECTOR(2 downto 0) := "000";
    signal Q   : STD_LOGIC_VECTOR(2 downto 0);

begin

    UUT : Register_PC
        port map (Clk => Clk, Res => Res, D => D, Q => Q);

    Clk <= not Clk after 25 ns;

    process
    begin
        -- Test 1: Async reset fires immediately without waiting for clock
        D <= "111"; Res <= '1';
        wait for 5 ns;

        -- Test 2: Hold reset, data ignored
        wait for 50 ns;

        -- Test 3: Release reset, load PC=1
        Res <= '0'; D <= "001";
        wait for 50 ns;

        -- Test 4: Sequential increment simulation
        D <= "010"; wait for 50 ns;
    end process;
end architecture;

```

```

D <= "011"; wait for 50 ns;
D <= "100"; wait for 50 ns;

-- Test 5: Jump target load
D <= "110"; wait for 50 ns;

-- Test 6: Async reset mid-execution
D <= "111";
wait for 3 ns;
Res <= '1';
wait for 2 ns;

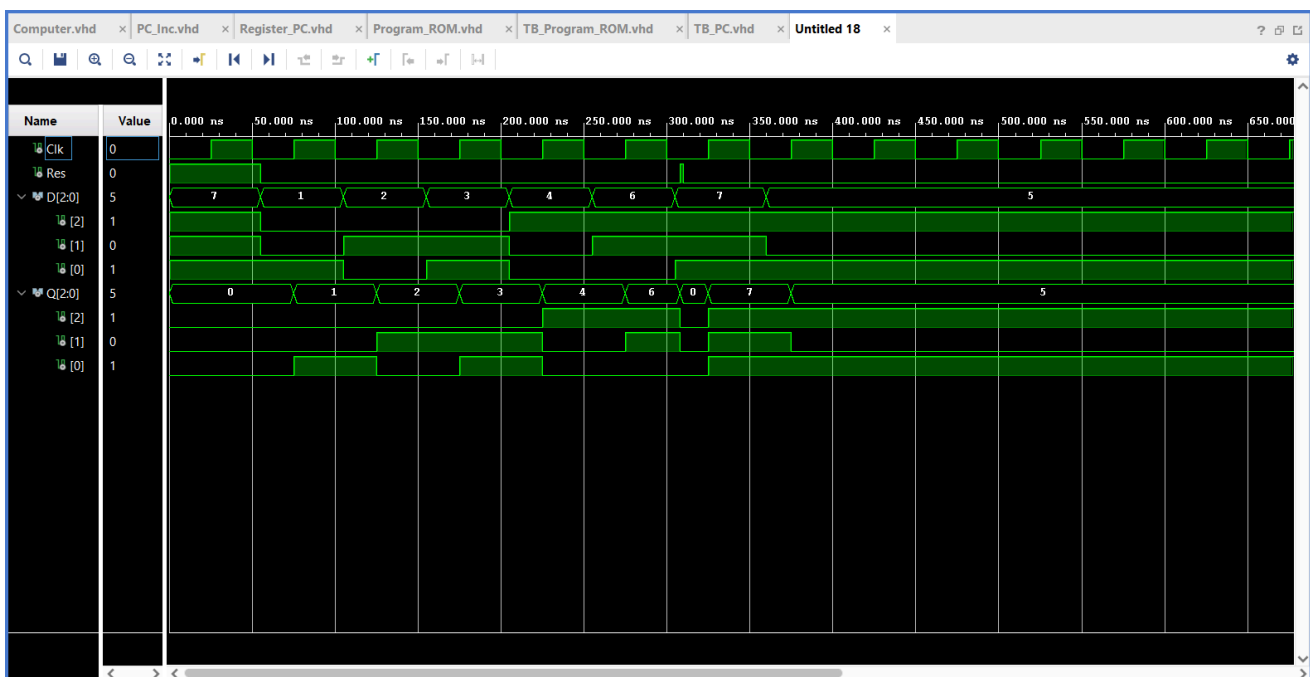
-- Test 7: Resume after reset
Res <= '0'; wait for 50 ns;
D <= "101"; wait for 50 ns;

    wait;
end process;

end Behavioral;

```

15.3. Timing Diagram



16. Program Incrementer

16.1. VHDL Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity PC_Inc is
    Port ( Q : in STD_LOGIC_VECTOR (2 downto 0);
          D : out STD_LOGIC_VECTOR (2 downto 0));
end PC_Inc;

architecture Behavioral of PC_Inc is
begin
    D <= std_logic_vector(unsigned(Q) + 1);
end Behavioral;
```

16.2. Testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_PC_Inc is
end TB_PC_Inc;

architecture Behavioral of TB_PC_Inc is

    component PC_Inc is
        Port ( Q : in STD_LOGIC_VECTOR(2 downto 0);
              D : out STD_LOGIC_VECTOR(2 downto 0));
    end component;

    signal Q : STD_LOGIC_VECTOR(2 downto 0) := "000";
    signal D : STD_LOGIC_VECTOR(2 downto 0);

begin

    UUT : PC_Inc port map (Q => Q, D => D);
```

```

process
begin
    -- Test all 8 PC values
    Q <= "000"; wait for 100 ns;
    Q <= "001"; wait for 100 ns;
    Q <= "010"; wait for 100 ns;
    Q <= "011"; wait for 100 ns;
    Q <= "100"; wait for 100 ns;
    Q <= "101"; wait for 100 ns;
    Q <= "110"; wait for 100 ns;

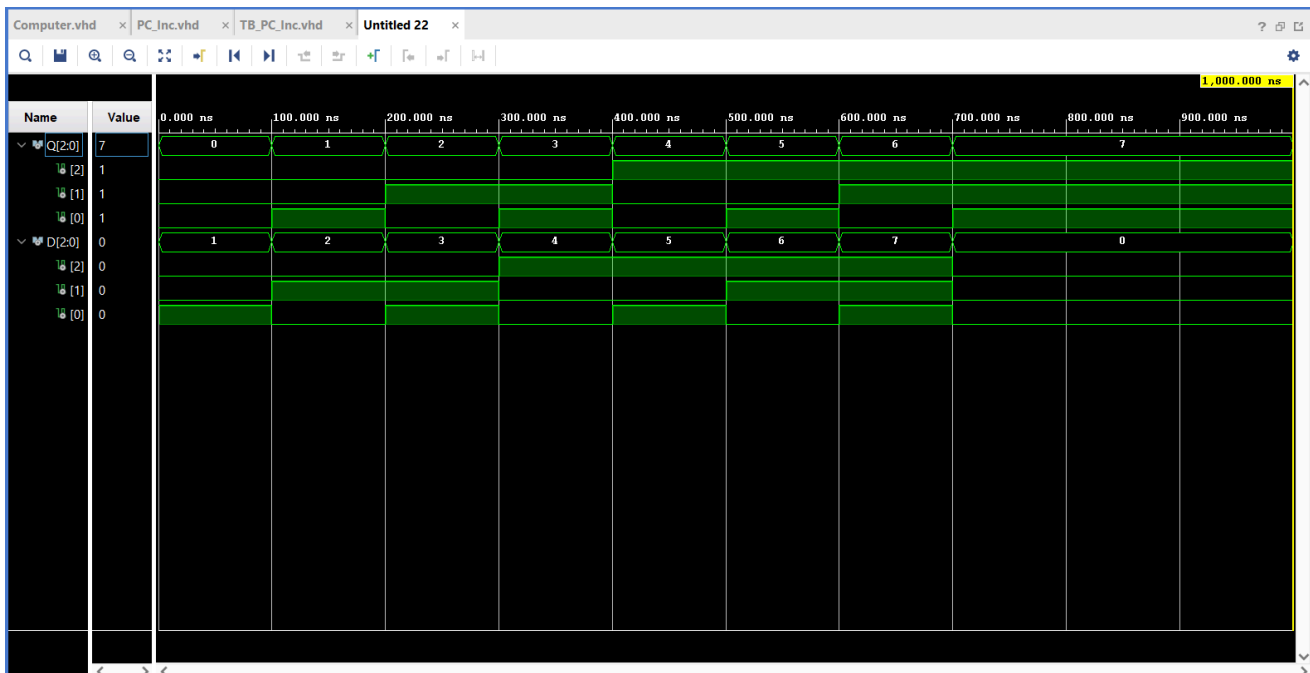
    -- Wrap-around: 7 + 1 = 0 (3-bit overflow)
    Q <= "111"; wait for 100 ns;

    wait;
end process;

end Behavioral;

```

16.3. Timing Diagram



17. Load Selector

17.1. VHDL Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Load_Selector is
    Port (
        A  : in STD_LOGIC_VECTOR (3 downto 0);
        B  : in STD_LOGIC_VECTOR (3 downto 0);
        Sel : in STD_LOGIC;
        O   : out STD_LOGIC_VECTOR (3 downto 0)
    );
end Load_Selector;

architecture Behavioral of Load_Selector is
begin
    process(A, B, Sel)
    begin
        if Sel = '0' then
            O <= A;
        else
            O <= B;
        end if;
    end process;
end Behavioral;
```

17.2. Testbench

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_LS is
end TB_LS;

architecture Behavioral of TB_LS is

    component Load_Selector is
        Port ( A      : in  STD_LOGIC_VECTOR(3 downto 0);
              B      : in  STD_LOGIC_VECTOR(3 downto 0);
              Sel     : in  STD_LOGIC;
              O       : out STD_LOGIC_VECTOR(3 downto 0));
    end component;

    signal A, B : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
    signal Sel  : STD_LOGIC := '0';
    signal O    : STD_LOGIC_VECTOR(3 downto 0);

begin

    UUT : Load_Selector port map (A => A, B => B, Sel => Sel, O => O);

    process
    begin
        -- ---- Test 1: Sel=0 → ALU result (A) ----
        A <= "1010"; B <= "0101"; Sel <= '0'; wait for 100 ns;

        -- ---- Test 2: Sel=1 → Immediate value (B) ----
        Sel <= '1'; wait for 100 ns;

        -- ---- Test 3: MOVI R7,3 → immediate=0011 ----
        A <= "1111"; B <= "0011"; Sel <= '1'; wait for 100 ns;

        -- ---- Test 4: ADD result → ALU path ----
        A <= "0111"; B <= "0000"; Sel <= '0'; wait for 100 ns;

        -- ---- Test 5: Zero immediate ----
        A <= "1111"; B <= "0000"; Sel <= '1'; wait for 100 ns;

        -- ---- Test 6: Max immediate value 15 ----
        B <= "1111"; Sel <= '1'; wait for 100 ns;
    end process;
end architecture;

```

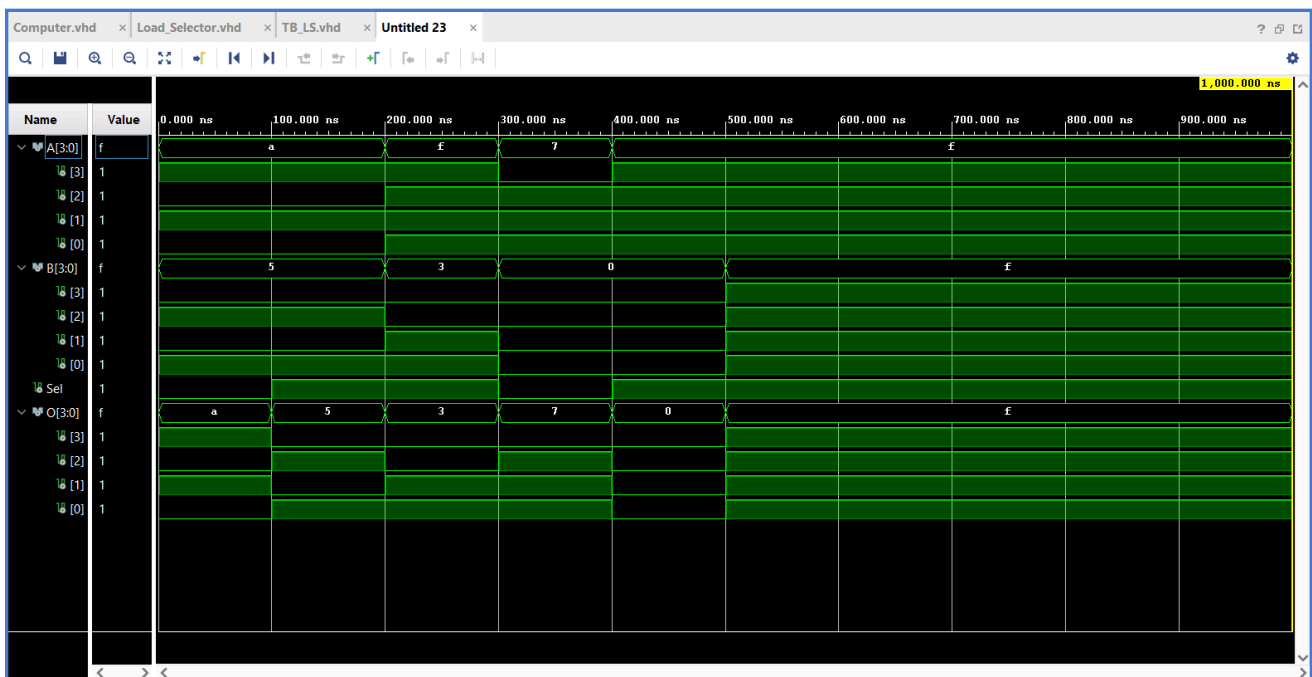
```

        wait;
    end process;

end Behavioral;

```

17.3. Timing Diagram



18. Instruction Decoder

18.1. VHDL Code

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Instruction_Decoder is
    Port (
        Instruction : in  STD_LOGIC_VECTOR(11 downto 0);
        JumpCheck   : in  STD_LOGIC_VECTOR(3 downto 0);
        RegEn       : out STD_LOGIC_VECTOR(2 downto 0);
        Sel_A       : out STD_LOGIC_VECTOR(2 downto 0);
        Sel_B       : out STD_LOGIC_VECTOR(2 downto 0);
        LoadSel     : out STD_LOGIC;
        ImmVal      : out STD_LOGIC_VECTOR(3 downto 0);
        Sub         : out STD_LOGIC;
        JumpFlag    : out STD_LOGIC;
        JumpAddress : out STD_LOGIC_VECTOR(2 downto 0));
end Instruction_Decoder;

architecture Behavioral of Instruction_Decoder is
begin
    process(Instruction, JumpCheck)
    begin
        -- Defaults prevent latches
        RegEn      <= (others => '0');
        Sel_A      <= (others => '0');
        Sel_B      <= (others => '0');
        LoadSel    <= '0';
        ImmVal     <= (others => '0');
        Sub        <= '0';
        JumpFlag   <= '0';
        JumpAddress <= (others => '0');

        case Instruction(11 downto 10) is
            when "00" => -- ADD: RegA = RegA + RegB
                RegEn <= Instruction(9 downto 7);
                Sel_A <= Instruction(9 downto 7);
                Sel_B <= Instruction(6 downto 4);

            when "01" => -- NEG: RegA = 0 - RegA (uses R0 as zero source)

```

```

        RegEn <= Instruction(9 downto 7);
        Sel_B <= Instruction(9 downto 7);
        Sub    <= '1';

    when "10" => -- MOVI: RegA = immediate
        RegEn    <= Instruction(9 downto 7);
        LoadSel  <= '1';
        ImmVal   <= Instruction(3 downto 0);

    when "11" => -- JZR: jump if register = 0
        Sel_A      <= Instruction(9 downto 7);
        JumpAddress <= Instruction(2 downto 0);
        if JumpCheck = "0000" then
            JumpFlag <= '1';
        end if;

    when others => null;
end case;

end process;
end Behavioral;

```

18.2. Testbench

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_IR is
end TB_IR;

architecture Behavioral of TB_IR is

    component Instruction_Decoder is
        Port ( Instruction : in  STD_LOGIC_VECTOR(11 downto 0);
              JumpCheck   : in  STD_LOGIC_VECTOR(3 downto 0);
              RegEn       : out STD_LOGIC_VECTOR(2 downto 0);
              Sel_A       : out STD_LOGIC_VECTOR(2 downto 0);
              Sel_B       : out STD_LOGIC_VECTOR(2 downto 0);
              LoadSel     : out STD_LOGIC;
              ImmVal      : out STD_LOGIC_VECTOR(3 downto 0);
              Sub         : out STD_LOGIC;
              JumpFlag    : out STD_LOGIC;
        );
    end component;

    -- Testbench logic would go here

```

```

        JumpAddress : out STD_LOGIC_VECTOR(2 downto 0));
end component;

signal Instruction : STD_LOGIC_VECTOR(11 downto 0) := (others => '0');
signal JumpCheck  : STD_LOGIC_VECTOR(3 downto 0)  := (others => '0');
signal RegEn      : STD_LOGIC_VECTOR(2 downto 0);
signal Sel_A      : STD_LOGIC_VECTOR(2 downto 0);
signal Sel_B      : STD_LOGIC_VECTOR(2 downto 0);
signal LoadSel    : STD_LOGIC;
signal ImmVal     : STD_LOGIC_VECTOR(3 downto 0);
signal Sub        : STD_LOGIC;
signal JumpFlag    : STD_LOGIC;
signal JumpAddress : STD_LOGIC_VECTOR(2 downto 0);

begin

    UUT : Instruction_Decoder
        port map (
            Instruction => Instruction,
            JumpCheck  => JumpCheck,
            RegEn      => RegEn,
            Sel_A      => Sel_A,
            Sel_B      => Sel_B,
            LoadSel    => LoadSel,
            ImmVal     => ImmVal,
            Sub        => Sub,
            JumpFlag    => JumpFlag,
            JumpAddress => JumpAddress
        );

    process
    begin
        -- Test 1a: ADD R7, R1
        Instruction <= "001110010000"; JumpCheck <= "0000"; wait for 60 ns;

        -- Test 1b: ADD R2, R1
        Instruction <= "000100010000"; JumpCheck <= "0000"; wait for 60 ns;

        -- Test 2a: NEG R1
        Instruction <= "010010000000"; JumpCheck <= "0000"; wait for 60 ns;

        -- Test 2b: NEG R7
        Instruction <= "011110000000"; JumpCheck <= "0000"; wait for 60 ns;

        -- Test 3a: MOVI R7, 3

```

```
Instruction <= "101110000011"; JumpCheck <= "0000"; wait for 60 ns;
```

```
-- Test 3b: MOVI R1, 1
```

```
Instruction <= "10010000001"; JumpCheck <= "0000"; wait for 60 ns;
```

```
-- Test 3c: MOVI R2, 3
```

```
Instruction <= "100100000011"; JumpCheck <= "0000"; wait for 60 ns;
```

```
-- Test 4a: JZR R0, 4 - JumpCheck=0 -> should jump
```

```
Instruction <= "110000000100"; JumpCheck <= "0000"; wait for 60 ns;
```

```
-- Test 4b: JZR R2, 6 - JumpCheck=0 -> should jump
```

```
Instruction <= "110100000110"; JumpCheck <= "0000"; wait for 60 ns;
```

```
-- Test 4c: JZR R2, 6 - JumpCheck non-zero -> no jump
```

```
Instruction <= "110100000110"; JumpCheck <= "0010"; wait for 60 ns;
```

```
-- Test 4d: JZR R7, 2 - JumpCheck non-zero -> no jump
```

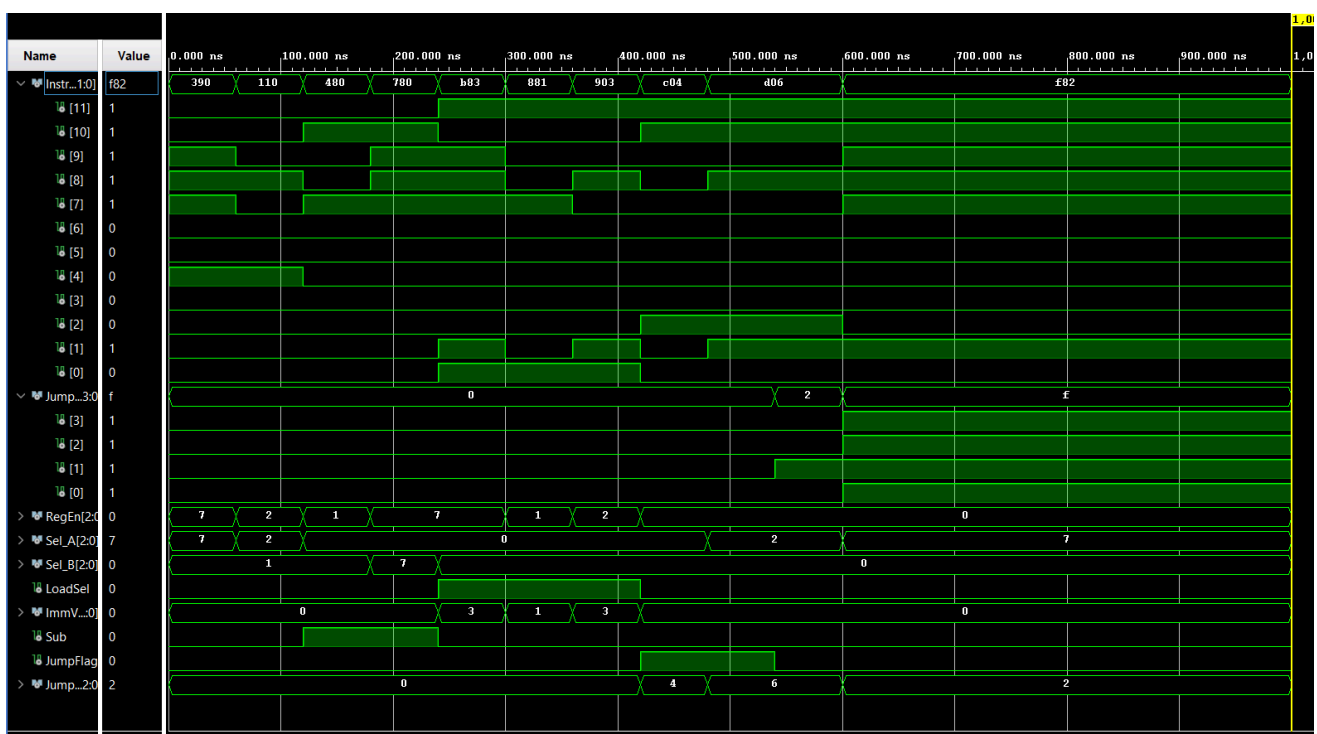
```
Instruction <= "111110000010"; JumpCheck <= "1111"; wait for 60 ns;
```

```
wait;
```

```
end process;
```

```
end Behavioral;
```

18.3. Timing Diagram



19. Segment Display

19.1. VHDL Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Display_Driver is
    Port ( Address      : in  STD_LOGIC_VECTOR(3 downto 0);
          driver_signal : out STD_LOGIC_VECTOR(6 downto 0));
end Display_Driver;

architecture Behavioral of Display_Driver is

    type rom_type is array (0 to 15) of STD_LOGIC_VECTOR(6 downto 0);
    constant sevenSegment_ROM : rom_type := (
        "1000000", -- 0
        "1111001", -- 1
        "0100100", -- 2
        "0110000", -- 3
        "0011001", -- 4
        "0010010", -- 5
        "0000010", -- 6
        "1111000", -- 7
        "0000000", -- 8
        "0010000", -- 9
        "0001000", -- A
        "0000011", -- B
        "1000110", -- C
        "0100001", -- D
        "0000110", -- E
        "0001110"  -- F
    );

begin
    driver_signal <= sevenSegment_ROM(to_integer(unsigned(Address)));
end Behavioral;
```


19.2. Testbench

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_7Seg_Dis is
end TB_7Seg_Dis;

architecture Behavioral of TB_7Seg_Dis is

    component Display_Driver is
        Port ( Address      : in  STD_LOGIC_VECTOR(3 downto 0);
              driver_signal : out STD_LOGIC_VECTOR(6 downto 0));
    end component;

    signal Address      : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
    signal driver_signal : STD_LOGIC_VECTOR(6 downto 0);

begin

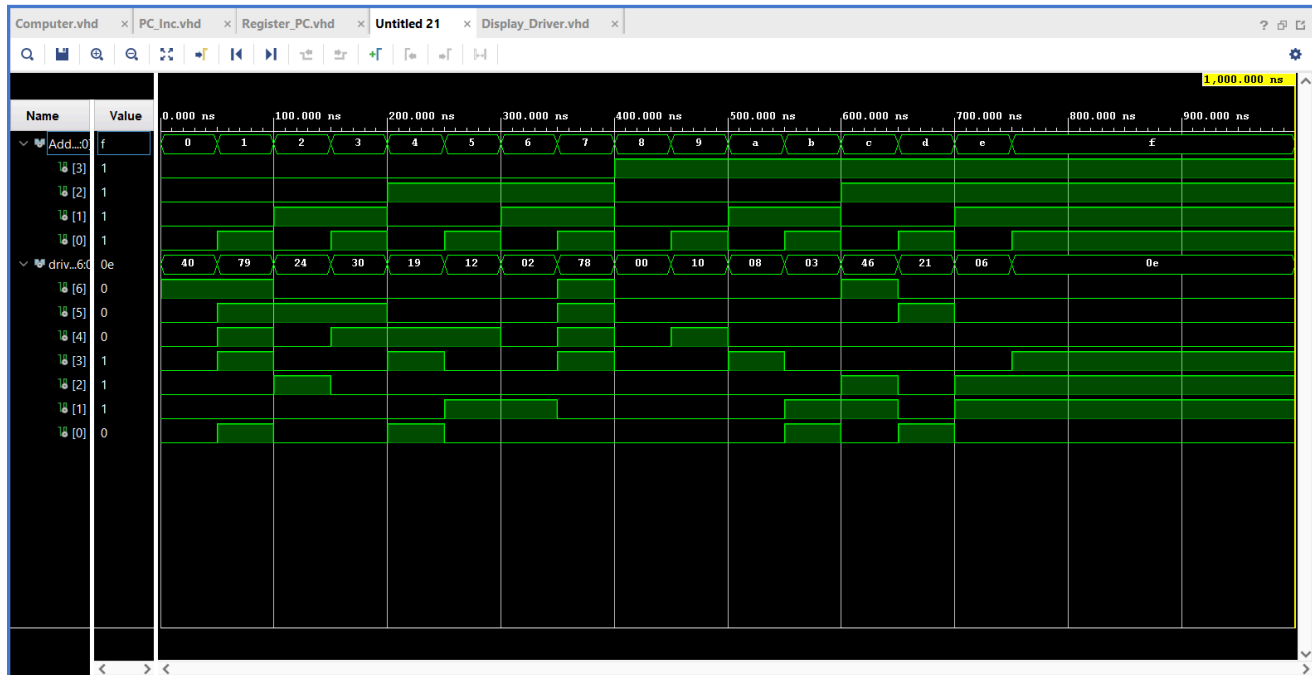
    UUT : Display_Driver port map (Address => Address, driver_signal =>
driver_signal);

    process
    begin
        Address <= "0000"; wait for 50 ns;  -- 0
        Address <= "0001"; wait for 50 ns;  -- 1
        Address <= "0010"; wait for 50 ns;  -- 2
        Address <= "0011"; wait for 50 ns;  -- 3
        Address <= "0100"; wait for 50 ns;  -- 4
        Address <= "0101"; wait for 50 ns;  -- 5
        Address <= "0110"; wait for 50 ns;  -- 6
        Address <= "0111"; wait for 50 ns;  -- 7
        Address <= "1000"; wait for 50 ns;  -- 8
        Address <= "1001"; wait for 50 ns;  -- 9
        Address <= "1010"; wait for 50 ns;  -- A
        Address <= "1011"; wait for 50 ns;  -- B
        Address <= "1100"; wait for 50 ns;  -- C
        Address <= "1101"; wait for 50 ns;  -- D
        Address <= "1110"; wait for 50 ns;  -- E
        Address <= "1111"; wait for 50 ns;  -- F
        wait;
    end process;

```

```
end Behavioral;
```

19.3. Timing Diagram



20. Processor

20.1. VHDL Code

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Processor is
    Port ( Reset      : in STD_LOGIC;
          Clk         : in STD_LOGIC;
          Overflow    : out std_logic;
          Zero        : out std_logic;
          R7 : out STD_LOGIC_VECTOR (3 downto 0)
        );
end Processor;

architecture Behavioral of Processor is

    Component Instruction_Decoder is
        Port (
            Instruction    : in  STD_LOGIC_VECTOR(11 downto 0);
            JumpCheck      : in  STD_LOGIC_VECTOR(3 downto 0);
            RegEn          : out STD_LOGIC_VECTOR(2 downto 0);
            Sel_A          : out STD_LOGIC_VECTOR(2 downto 0);
            Sel_B          : out STD_LOGIC_VECTOR(2 downto 0);
            LoadSel        : out STD_LOGIC;
            ImmVal         : out STD_LOGIC_VECTOR(3 downto 0);
            Sub             : out STD_LOGIC;
            JumpFlag       : out STD_LOGIC;
            JumpAddress    : out STD_LOGIC_VECTOR(2 downto 0)
        );
    end Component;

    Component MUX8 is
        Port (
            Sel : in  STD_LOGIC_VECTOR(2 downto 0);
            I0  : in  STD_LOGIC_VECTOR(3 downto 0);
            I1  : in  STD_LOGIC_VECTOR(3 downto 0);
            I2  : in  STD_LOGIC_VECTOR(3 downto 0);
            I3  : in  STD_LOGIC_VECTOR(3 downto 0);
            I4  : in  STD_LOGIC_VECTOR(3 downto 0);
            I5  : in  STD_LOGIC_VECTOR(3 downto 0);
            I6  : in  STD_LOGIC_VECTOR(3 downto 0);
            I7  : in  STD_LOGIC_VECTOR(3 downto 0);

```

```

        O      : out STD_LOGIC_VECTOR(3 downto 0)
    );
end Component;

Component MUX2 is
    port(
        Sel : in std_logic;
        A   : in std_logic_vector(2 downto 0);
        B   : in std_logic_vector(2 downto 0);
        O   : out std_logic_vector(2 downto 0)
    );
end Component;

Component Load_Selector is
    Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
          B : in STD_LOGIC_VECTOR (3 downto 0);
          Sel : in STD_LOGIC;
          O : out STD_LOGIC_VECTOR (3 downto 0));
end Component;

Component PC_Inc is
    Port ( Q : in STD_LOGIC_VECTOR (2 downto 0);
          D : out STD_LOGIC_VECTOR (2 downto 0));
end Component;

Component Program_ROM is
    port(ROM_address : in STD_LOGIC_VECTOR (2 downto 0);
          I: out STD_LOGIC_VECTOR (11 downto 0)
    );
end Component;

Component Register_PC is
    Port (
        Clk      : in STD_LOGIC;
        Res      : in STD_LOGIC;
        D        : in STD_LOGIC_VECTOR(2 downto 0);
        Q        : out STD_LOGIC_VECTOR(2 downto 0)
    );
end Component;

Component AU is
    Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
          B : in STD_LOGIC_VECTOR (3 downto 0);
          Sub : in STD_LOGIC;
          S : out STD_LOGIC_VECTOR (3 downto 0);

```

```

        Overflow : out STD_LOGIC;
        Zero : out STD_LOGIC);
end Component;

Component Register_Bank is
PORT (
    RegEn      : IN  STD_LOGIC_VECTOR(2 DOWNTO 0);
    Data       : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
    Clk        : IN  STD_LOGIC;
    Res        : IN  STD_LOGIC;
    Reg0       : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
    Reg1       : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
    Reg2       : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
    Reg3       : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
    Reg4       : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
    Reg5       : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
    Reg6       : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
    Reg7       : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
);
END Component;

-----

signal Next_Address      : STD_LOGIC_VECTOR(2 DOWNTO 0);
signal Current_Address   : STD_LOGIC_VECTOR(2 DOWNTO 0);
signal Selected_Address  : STD_LOGIC_VECTOR(2 DOWNTO 0);
signal Jump_Address      : STD_LOGIC_VECTOR(2 DOWNTO 0);
signal Jump_Flag         : std_logic;
signal Instruction       : STD_LOGIC_VECTOR(11 DOWNTO 0);

signal t0, t1, t2, t3, t4, t5, t6, t7 : STD_LOGIC_VECTOR(3 DOWNTO 0);

signal Load_Selection    : std_logic;
signal Immediate_Value   : STD_LOGIC_VECTOR(3 DOWNTO 0);
signal ASelect           : STD_LOGIC_VECTOR(2 DOWNTO 0);
signal BSelect           : STD_LOGIC_VECTOR(2 DOWNTO 0);
signal AData              : STD_LOGIC_VECTOR(3 DOWNTO 0);
signal BData              : STD_LOGIC_VECTOR(3 DOWNTO 0);
signal Operation_Result  : STD_LOGIC_VECTOR(3 DOWNTO 0);
signal Sub_Sel           : std_logic;
signal Register_Enable   : STD_LOGIC_VECTOR(2 DOWNTO 0);
signal Selected_Load     : STD_LOGIC_VECTOR(3 DOWNTO 0);

```

```

-----

begin

Program_Counter : Register_PC
Port Map (
    Clk => Clk,
    Res => Reset,
    D => Selected_Address,
    Q => Current_Address
);

Incrementer : PC_Inc
Port Map (
    Q => Current_Address,
    D => Next_Address
);

Next_Address_Selector : MUX2
Port Map (
    Sel => Jump_Flag,
    A => Next_Address,
    B => Jump_Address,
    O => Selected_Address
);

InstructionDecoder : Instruction_Decoder
Port Map (
    Instruction => Instruction,
    JumpCheck  => AData,
    RegEn      => Register_Enable,
    Sel_A      => ASelect,
    Sel_B      => BSelect,
    LoadSel   => Load_Selection,
    ImmVal     => Immediate_Value,
    Sub        => Sub_Sel,
    JumpFlag   => Jump_Flag,
    JumpAddress => Jump_Address
);

LoadSelector : Load_Selector
Port Map (
    A => Operation_Result,
    B => Immediate_Value,

```

```

    Sel => Load_Selection,
    0   => Selected_Load
);

```

```

RegisterBank : Register_Bank

```

```

    Port Map (
        RegEn    => Register_Enable,
        Data      => Selected_Load,
        Clk       => Clk,
        Res       => Reset,
        Reg0      => t0,
        Reg1      => t1,
        Reg2      => t2,
        Reg3      => t3,
        Reg4      => t4,
        Reg5      => t5,
        Reg6      => t6,
        Reg7      => t7
    );

```

```

Selector_A : MUX8

```

```

    Port Map (
        Sel    => ASelect,
        I0     => t0,
        I1     => t1,
        I2     => t2,
        I3     => t3,
        I4     => t4,
        I5     => t5,
        I6     => t6,
        I7     => t7,
        0      => AData
    );

```

```

Selector_B : MUX8

```

```

    Port Map (
        Sel    => BSelect,
        I0     => t0,
        I1     => t1,
        I2     => t2,
        I3     => t3,
        I4     => t4,
        I5     => t5,
        I6     => t6,

```

```

        I7    => t7,
        0     => BData
    );

Arithmetic_Unit : AU
    Port Map(
        A    => AData,
        B    => BData,
        Sub   => Sub_Sel,
        S     => Operation_Result,
        Overflow => Overflow,
        Zero  => Zero
    );

ProgramROM : Program_ROM
    Port Map(
        ROM_address => Current_Address,
        I           => Instruction
    );

R7 <= t7;

end Behavioral;

```

20.2. Testbench

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

-- =====
-- Testbench : TB_NanoProcessor
-- DUT       : Processer (complete processor)
--
-- Loaded program (from Program_ROM):
-- Addr 0: MOVI R7, 3    → R7 = 3
-- Addr 1: MOVI R1, 1    → R1 = 1
-- Addr 2: NEG  R1       → R1 = -1 (0xF = 15)
-- Addr 3: MOVI R2, 3    → R2 = 3
-- Addr 4: ADD  R2, R1    → R2 = R2 + R1 = 3 + 15 = 2 (wraps) → R2 = 2
-- Addr 5: ADD  R7, R2    → R7 = R7 + R2 = 3 + 2 = 5

```



```

-- Addr 6: JZR R2, 6 → R2 != 0 → no jump → PC = 7
-- Addr 7: JZR R0, 4 → R0 = 0 → jump to 4 (always)
--
-- Expected R7 progression: 3 → 5 → 7 → 9 → 11 → ... (+2 each loop)
-- =====
entity TB_Processor is
end TB_Processor;

architecture Behavioral of TB_Processor is

    component Processer is
        Port ( Reset      : in  STD_LOGIC;
              Clk         : in  STD_LOGIC;
              Overflow    : out STD_LOGIC;
              Zero        : out STD_LOGIC;
              R7          : out STD_LOGIC_VECTOR(3 downto 0));
    end component;

    signal Clk         : STD_LOGIC := '0';
    signal Reset       : STD_LOGIC := '1';
    signal Overflow    : STD_LOGIC;
    signal Zero        : STD_LOGIC;
    signal R7          : STD_LOGIC_VECTOR(3 downto 0);

    constant CLK_PERIOD : time := 20 ns;

begin

    UUT : Processer
        port map (Reset => Reset, Clk => Clk,
                  Overflow => Overflow, Zero => Zero, R7 => R7);

    Clk <= not Clk after CLK_PERIOD / 2;

    process
    begin
        -- ---- Apply Reset ----
        Reset <= '1';
        wait for CLK_PERIOD * 2;
        Reset <= '0';
        wait for CLK_PERIOD / 2; -- align to just after rising edge

        -- ---- Cycle 0: MOVI R7, 3 ----
        wait for CLK_PERIOD;
        -- ---- Cycle 1: MOVI R1, 1 ----

```

```

wait for CLK_PERIOD;
-- ---- Cycle 2: NEG R1 ----
wait for CLK_PERIOD;
-- ---- Cycle 3: MOVI R2, 3 ----
wait for CLK_PERIOD;
-- ---- Cycle 4: ADD R2, R1 (R2 = 3 + NEG(1) = 3-1 = 2) ----
wait for CLK_PERIOD;
-- ---- Cycle 5: ADD R7, R2 (R7 = 3 + 2 = 5) ----
wait for CLK_PERIOD;
wait for CLK_PERIOD / 2;

-- ---- Cycle 6: JZR R2, 6 (R2=2 /= 0, no jump, PC→7) ----
wait for CLK_PERIOD;
-- ---- Cycle 7: JZR R0, 4 (R0=0, always jump to 4) ----
wait for CLK_PERIOD;

-- ---- Second iteration: ADD R2, R1 again → R2 wraps differently ----
-- Addr 4: ADD R2, R1 → R2 = 2 + 15 = 1 (wrap)
-- Addr 5: ADD R7, R2 → R7 = 5 + 1 = 6 (but depends on R2)
-- Let the processor run several more loops
wait for CLK_PERIOD * 4;

-- ---- Test Reset mid-execution ----
Reset <= '1';
wait for CLK_PERIOD * 2;

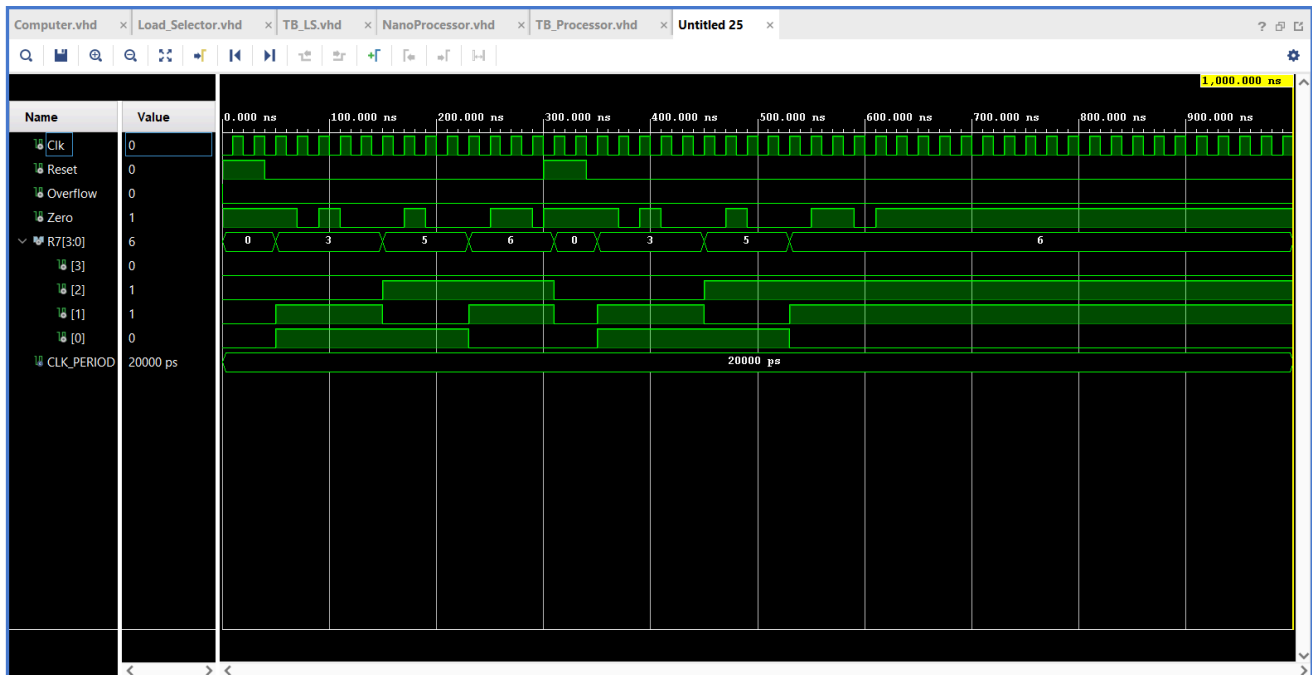
-- ---- Resume and check R7 loads again ----
Reset <= '0';
wait for CLK_PERIOD * 6; -- allow MOVI R7,3 and first ADD to execute

wait;
end process;

end Behavioral;

```

20.3. Timing Diagram



21. NanoProcessor

21.1. VHDL Code

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity NanoProcessor is
    Port ( Clk : in STD_LOGIC;
          Reset : in STD_LOGIC;
          Seg7 : out STD_LOGIC_VECTOR (6 downto 0);
          data : out STD_LOGIC_VECTOR (3 downto 0);
          Zero : out STD_LOGIC;
          Overflow : out STD_LOGIC;
          Anode: out STD_LOGIC_VECTOR (3 downto 0));
end NanoProcessor;

architecture Behavioral of NanoProcessor is

    Component Slow_Clk is
        generic (
            N : INTEGER := 2
        );
        Port (
            Clk_in : in STD_LOGIC;
            Clk_out : out STD_LOGIC
        );
    end Component;

    Component Display_Driver is
        Port ( Address : in STD_LOGIC_VECTOR (3 downto 0);
              driver_signal : out STD_LOGIC_VECTOR (6 downto 0)
        );
    end Component;

    Component Processer is
        Port ( Reset : in STD_LOGIC;
              Clk : in STD_LOGIC;
              Overflow : out std_logic;
              Zero : out std_logic;

              R7 : out STD_LOGIC_VECTOR (3 downto 0)
        );

```

```

end Component;

Signal R7 : STD_LOGIC_VECTOR (3 downto 0);
Signal SC : STD_LOGIC;

begin

    SlowClk : Slow_Clk
        generic map( N => 2 )
        port map(
            Clk_in  => Clk,
            Clk_out => SC
        );

    NP1 : Processor
        Port Map(
            Reset    => Reset,
            Clk       => SC,
            Overflow  => Overflow,
            Zero      => Zero,
            R7        => R7
        );

    Driver : Display_Driver
        Port Map(
            Address  => R7,
            driver_signal => Seg7
        );

    data <= R7;
    Anode <= "1110";

end Behavioral;

```

21.2. Testbench

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```

entity TB_NanoProcessor is
end TB_NanoProcessor;

architecture Behavioral of TB_NanoProcessor is

    component NanoProcessor is
    Port ( Clk : in STD_LOGIC;
          Reset : in STD_LOGIC;
          Seg7 : out STD_LOGIC_VECTOR (6 downto 0);
          data : out STD_LOGIC_VECTOR (3 downto 0);
          Zero : out STD_LOGIC;
          Overflow : out STD_LOGIC;
          Anode : out STD_LOGIC_VECTOR(3 downto 0));
    end component;

    -- Internal signals
    signal ClkIn : std_logic := '0';
    signal ResetIn : std_logic := '0';
    signal Overflow : std_logic;
    signal Zero : std_logic;
    signal seg : std_logic_vector(6 downto 0);
    signal led : std_logic_vector(3 downto 0);
    signal Anode : STD_LOGIC_VECTOR(3 downto 0);

begin

    -- Instantiate the Unit Under Test (UUT)
    uut: NanoProcessor
        port map (
            Clk => ClkIn,
            Reset => ResetIn,
            Seg7 => seg,
            data => led,
            Zero => Zero,
            Overflow => Overflow,
            Anode => Anode
        );

    Reset_process : process
    begin
        ResetIn <= '1'; -- assert reset
        wait for 20 ns;
        ResetIn <= '0'; -- release reset
        wait;
    end process;

```

```

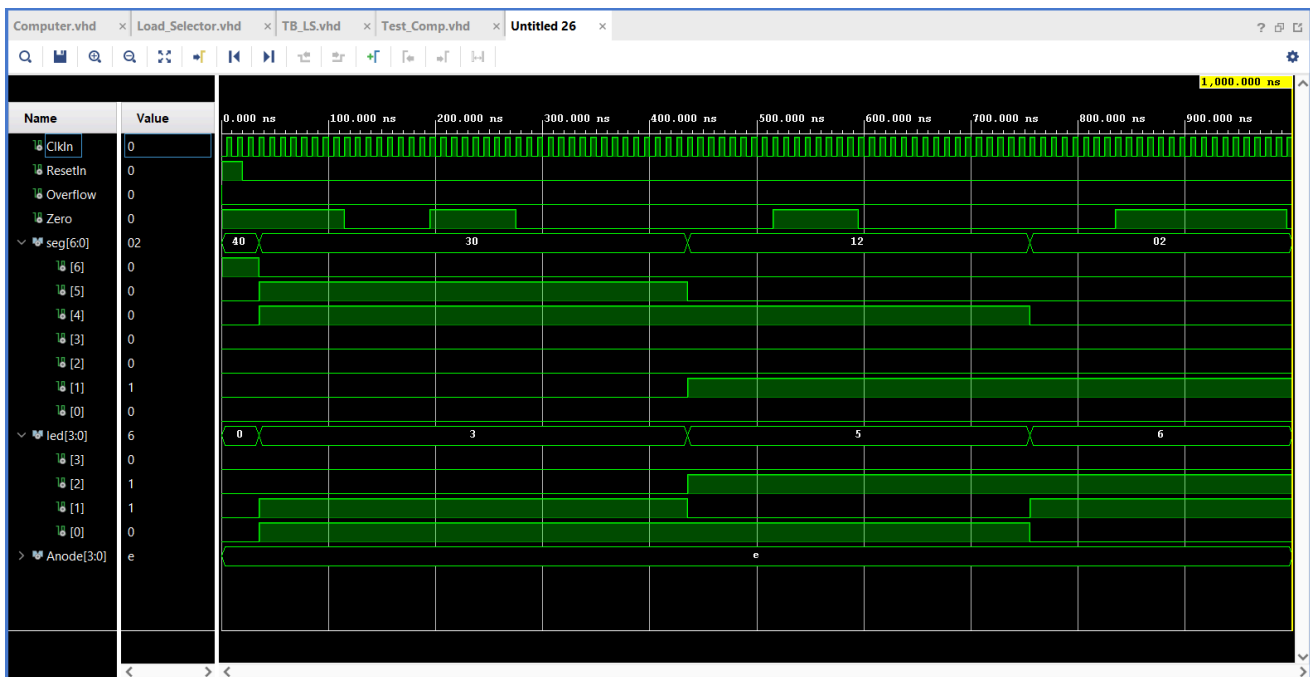
end process;

Clk_process : process
begin
    while true loop
        ClkIn <= '0';
        wait for 5 ns;
        ClkIn <= '1';
        wait for 5 ns;
    end loop;
end process;

end Behavioral;

```

21.3. Timing Diagram



22. Resource Optimization Strategies

In this project, we focused on making our Nanoprocessor as "lean" as possible. Our main goal was to reduce the number of Look-Up Tables (LUTs) used, which are the basic building blocks of the FPGA. By minimizing these, we ensured the design runs faster and uses fewer hardware resources. Below are the specific strategies we applied:

❖ Using Dedicated Carry Logic

Instead of writing a generic math formula and letting the software guess how to build it, we specifically used the CARRY4 component in our `adder_4bit.vhd` file.

- The Problem: Normally, standard code forces the FPGA to use general-purpose LUTs to handle mathematical "carries," which can be slow and wasteful.
- Our Solution: By calling the high-speed carry chains already built into the silicon, we freed up those LUTs for other tasks and significantly boosted the speed of our additions.

❖ Smart Component Reuse (The Unified AU)

In our Arithmetic Unit (`AU.vhd`), we made sure not to build two separate circuits for adding and subtracting.

- The Strategy: We implemented a single 4-bit adder that handles both operations.
- The Logic: When the "Subtract" signal is turned on, we use an XOR gate to flip the input bits and feed a "plus one" into the carry-in. This performs 2's complement subtraction using the same hardware as an addition, effectively cutting our logic requirements in half.

❖ Hardwiring Register Zero

To simplify the design, we permanently set Reg0 to zero ("0000") in the `Register_Bank.vhd`.

- Why we did it: Since R0 is a constant and can't be changed, the synthesis software (Vivado) realized it didn't need to create physical storage (flip-flops) for it.
- The Impact: This doesn't just save space for that one register; it also allows the system to simplify every multiplexer or gate connected to R0, stripping away unnecessary logic across the whole processor.

❖ Streamlined Instruction Decoding

We kept the `Instruction_Decoder.vhd` very efficient by having it look primarily at the top two bits of an instruction to decide what to do.

- Efficiency: By using a clean "case" statement, the decoder can identify commands like ADD or MOVI with very little logic.
- Direct Wiring: For tasks like routing register addresses, we used direct wire connections that don't require any LUTs at all, keeping the decoding process extremely fast.

❖ Optimized Multiplexer Structure

Finally, in our `MUX8.vhd` and `MUX2.vhd` files, we prioritized "flat" coding structures instead of using many "if-else" layers.

- FPGA Compatibility: Modern FPGAs are built to handle 6 input structures. By using flat case statements, we ensured our multiplexers fit perfectly into these hardware units.
- The Result: This stopped the software from creating extra "layers" of logic, meaning data moves through the multiplexers in a single, fast step.

23. Additional Features and Enhancements

Beyond the core requirements of the lab, we integrated several features to make the processor more functional and easier to control during testing. These additions focused on expanding the memory capacity and adding real-time manual control over the execution flow.

Hardware Pause Control (Manual Switch)

One of the most practical features we added is a physical "**Switch Operator**" that acts as a pause button for the entire system.

- How it works: When the switch is toggled ON, the program runs normally.
- The Pause Mechanism: If the switch is turned OFF while the program is in the middle of a task, the current instruction is immediately **frozen** or **paused**.
- Resuming Execution: We designed the system to hold its exact state meaning the Program Counter and all register values stay the same so that when the switch is flipped back on, the program continues from the exact point where it stopped.

Expanding the Instruction Set (ISA)

To make the processor more versatile for logic-heavy tasks, we expanded the ISA to include three fundamental bitwise operations:

- AND & OR: These allow for comparing and combining data between two registers.
- NOT: This allows for bitwise inversion of a single register's value.
- Technical Implementation: To get these working, we updated the Instruction Decoder to recognize these new commands and modified the Arithmetic Unit (AU) to process the logic.

Memory and Register Upgrades

To handle larger programs and more data, we scaled up the internal storage capacity:

- 16-Slot ROM: We expanded the program memory from 8 to 16 instructions. This involved upgrading the Program Counter (**PC**) to a 4-bit system to address the extra slots.
- 16-Register File: Instead of the standard 8 registers, we doubled the capacity to 16 registers (**R0 through R15**). This required a 4-bit selection line and a 16-to-1 Multiplexer (**MUX16**) to manage the data flow between them.

24. Team Members — Individual Contributions

The successful development of the Nano Processor was a collaborative effort, with each team member contributing significantly to specific components and aspects of the project. The details below outline the primary responsibilities and contributions of each member of Group 33.

UMAYANGA P.H.A. (240673E)

Designed the Instruction Decoder, assisted with module integration, system testing, debugging, and report preparation

UTHTHARASENA U.G.N.S. (240675L)

Implemented and tested the 4-bit Add/Subtract Unit (ALU), verified arithmetic operations, and contributed to report writing.

VITHANAGE K.O. (240688E)

Designed the Register Bank and Program Counter, integrated LEDs and 7-segment display outputs, tested the hardware implementation, and contributed to the final report. .

THISARUKA K.K.M. (240668T)

Developed the Program ROM, converted assembly instructions to machine code, tested program execution, and assisted with documentation.

25. Conclusion

In conclusion, this laboratory project successfully demonstrated the design and implementation of a simple 4-bit nanoprocessor capable of executing basic instructions such as MOVI, ADD, NEG, and JZR. Through the development of components including the arithmetic unit, register bank, multiplexers, instruction decoder, program counter, and ROM, a clear understanding of processor architecture and datapath operations was achieved. The project also provided practical experience in designing combinational and sequential logic circuits using VHDL and integrating them into a complete digital system. Simulation and hardware testing on the BASYS 3 FPGA board verified the correct functionality of the nanoprocessor and highlighted the importance of careful module integration and debugging during hardware development. Additionally, implementing an assembly program and converting it into machine code helped in understanding the relationship between software instructions and hardware execution. This lab further improved teamwork, communication, and problem-solving skills, as each member contributed to different parts of the project and collaborated during integration and testing. Overall, the experiment provided valuable hands-on experience in computer organization, processor design, FPGA implementation, and digital system development, forming a strong foundation for understanding more advanced computer architectures and embedded systems in the future.